

Instituto Politecnico Nacional

Escuela Superior de Computo

Practicas 7, 8 y 9

Operaciones del Algebra Relacional

Dominio seleccionado: Sistema universitario

Integrante: Julio Adrian Milan Huidobro

Unidad de aprendizaje: Bases de Datos

Programa academico: Ingenieria en Sistemas Computacionales

Fecha de entrega: martes 16 de junio de 2026

Objetivo del documento

Aplicar operaciones del algebra relacional, calculo relacional de tuplas, calculo relacional de dominios y SQL sobre un esquema universitario coherente. El mismo proyecto se usa para resolver los ejercicios de las practicas 7, 8 y 9 y el caso integrador final.

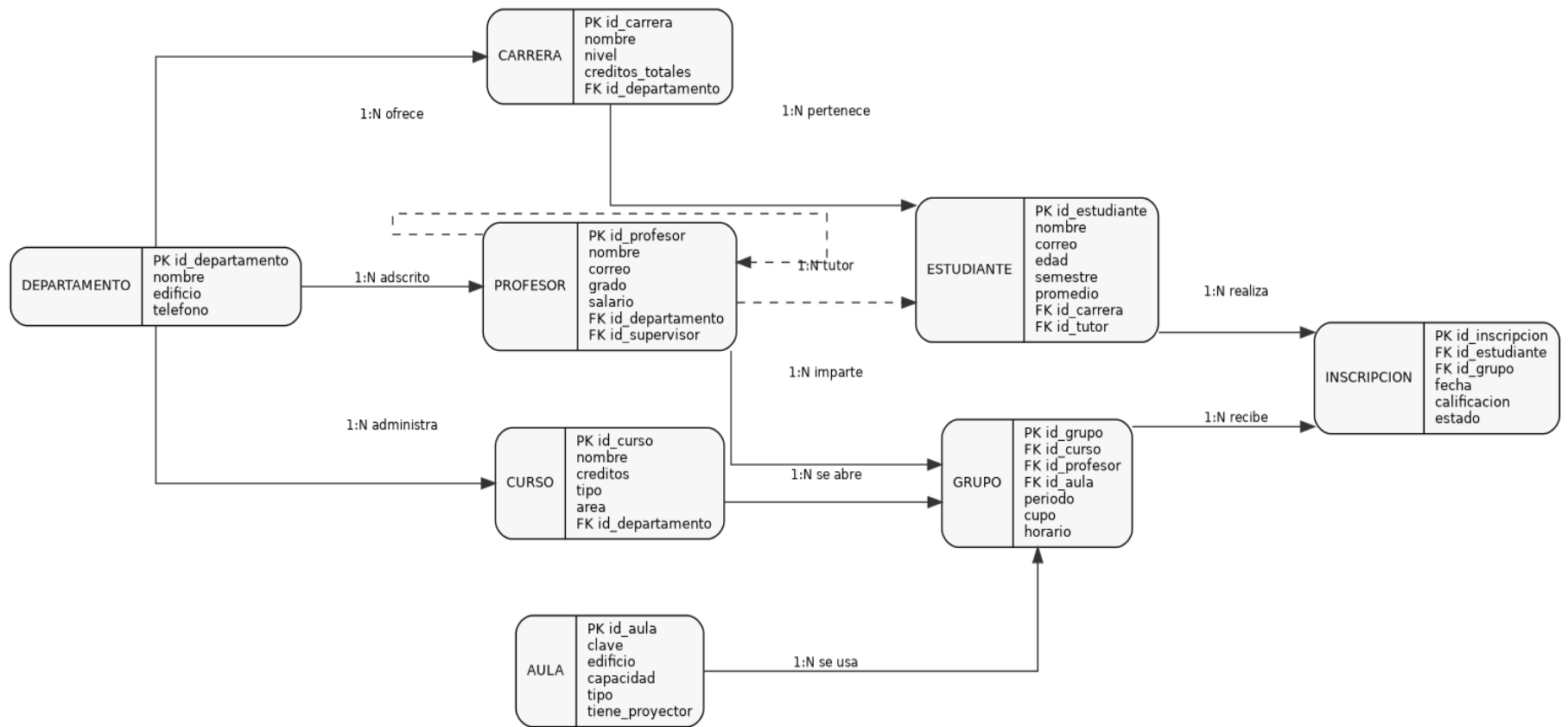
1. Descripcion del dominio

El dominio modela una universidad con departamentos academicos, carreras, profesores, estudiantes, cursos, aulas, grupos e inscripciones. Permite consultar trayectoria academica, oferta de cursos, asignacion docente, uso de aulas, calificaciones y relaciones jerarquicas entre profesores.

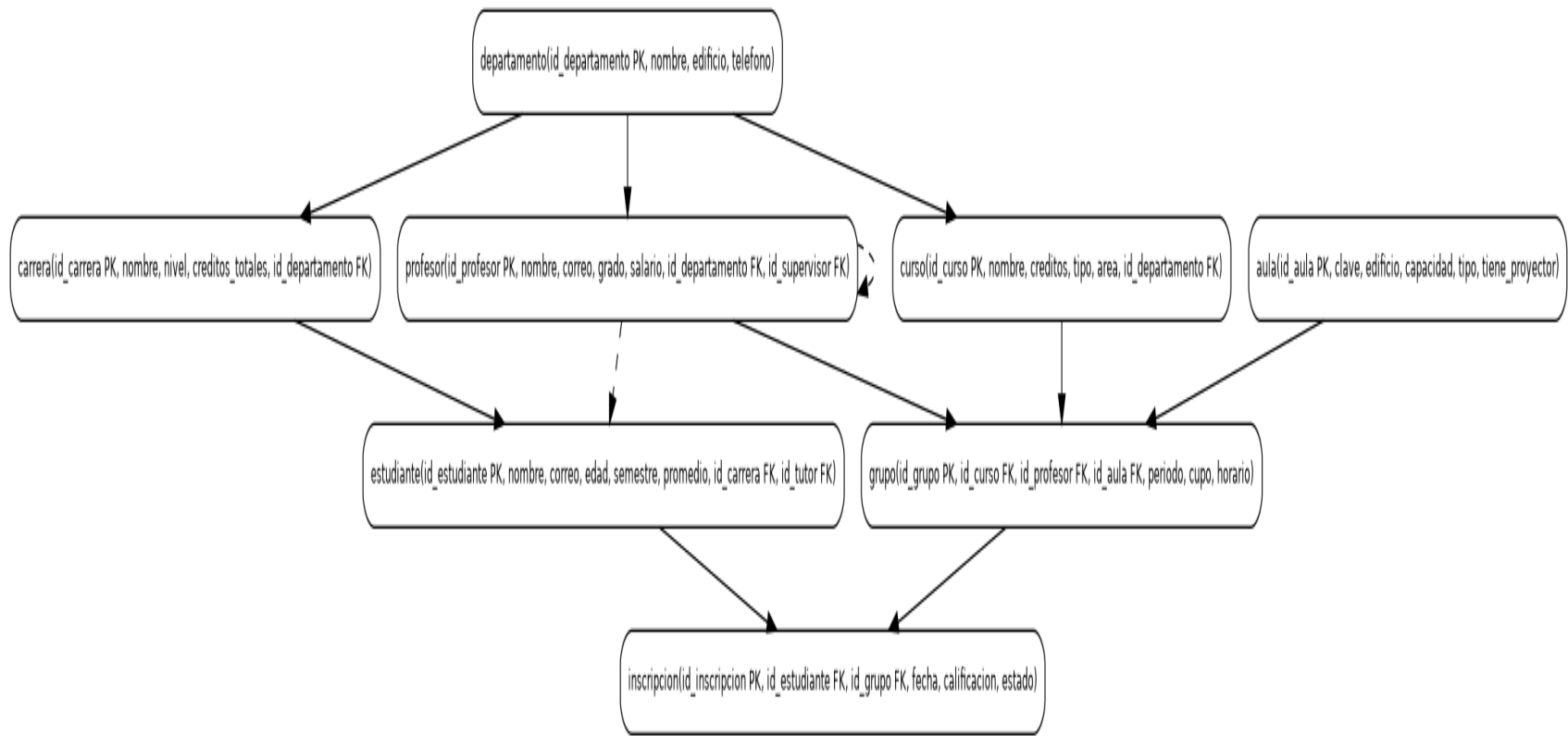
1.1 Relaciones propuestas

Relacion	Descripcion	Clave primaria	Claves foraneas
DEPARTAMENTO	Area academica responsable de carreras, cursos y profesores.	id_departamento	-
CARRERA	Programa academico al que pertenecen estudiantes.	id_carrera	id_departamento
PROFESOR	Docente adscrito a un departamento y posible supervisor.	id_profesor	id_departamento, id_supervisor
ESTUDIANTE	Alumno inscrito en una carrera y opcionalmente asignado a tutor.	id_estudiante	id_carrera, id_tutor
CURSO	Unidad de aprendizaje obligatoria u optativa.	id_curso	id_departamento
AULA	Espacio fisico donde se imparten grupos.	id_aula	-
GRUPO	Apertura de un curso en un periodo con profesor y aula.	id_grupo	id_curso, id_profesor, id_aula
INSCRIPCION	Relacion estudiante-grupo con fecha, calificacion y estado.	id_inscripcion	id_estudiante, id_grupo

1.2 Diagrama Entidad-Relacion Extendido (EER)



1.3 Modelo relacional



1.4 Cardinalidad de datos

Tabla	Tuplas
departamento	5
carrera	5
profesor	12
estudiante	20
curso	15
aula	10
grupo	20
inscripcion	102

Total de tuplas del proyecto: 189. Se cumple el requisito de mas de 100 tuplas distribuidas entre las tablas.

2. Parte 1 - Operadores basicos del Algebra Relacional

2.1 Ejercicio 1.1 - Esquema relacional y datos

El esquema se entrega en el archivo sql/01_schema.sql y los datos en sql/02_data.sql. A continuacion se muestra una sintesis de atributos por relacion.

Relacion	Atributos
DEPARTAMENTO	id_departamento, nombre, edificio, telefono
CARRERA	id_carrera, nombre, nivel, creditos_totales, id_departamento
PROFESOR	id_profesor, nombre, correo, grado, salario, id_departamento, id_supervisor
ESTUDIANTE	id_estudiante, nombre, correo, edad, semestre, promedio, id_carrera, id_tutor
CURSO	id_curso, nombre, creditos, tipo, area, id_departamento
AULA	id_aula, clave, edificio, capacidad, tipo, tiene_proyector
GRUPO	id_grupo, id_curso, id_profesor, id_aula, periodo, cupo, horario
INSCRIPCION	id_inscripcion, id_estudiante, id_grupo, fecha, calificacion, estado

2.2 Ejercicio 1.2 - Operador de seleccion (σ)

S1. Condicion simple con igualdad: estudiantes de la carrera ISC

Algebra relacional:

$\sigma_{id_carrera = 1}(ESTUDIANTE)$

SQL equivalente:

`SELECT id_estudiante, nombre, semestre, promedio FROM estudiante WHERE id_carrera = 1 ORDER BY id_estudiante;`

Resultado ejemplo:

id_estudiante	nombre	semestre	promedio
1	Julio Milan	5	9.1
2	Fernanda Lopez	6	8.8
3	Diego Martinez	5	8.6
11	Sebastian Morales	3	8.3
17	Bruno Salazar	8	6.9

S2. Condicion con desigualdad: cursos que no son optativos

Algebra relacional:

$\sigma_{tipo \neq 'Optativo'}(CURSO)$

SQL equivalente:

`SELECT id_curso, nombre, tipo FROM curso WHERE tipo <> 'Optativo' ORDER BY id_curso;`

Resultado ejemplo:

id_curso	nombre	tipo
1	Bases de Datos	Obligatorio
2	Algoritmos y Estructuras	Obligatorio
3	Calculo Vectorial	Obligatorio
4	Algebra Lineal	Obligatorio

id_curso	nombre	tipo
5	Redes de Computadoras	Obligatorio
6	Sistemas Operativos	Obligatorio
13	Arquitectura de Computadoras	Obligatorio
15	Etica Profesional	Obligatorio

S3. Comparacion: profesores con salario mayor a 40000

Algebra relacional:

$\sigma_{\text{salario} > 40000}(\text{PROFESOR})$

SQL equivalente:

SELECT id_profesor, nombre, salario FROM profesor WHERE salario > 40000 ORDER BY salario DESC;

Resultado ejemplo:

id_profesor	nombre	salario
12	Dra. Valeria Soto	46000.0
7	Dra. Elena Navarro	45000.0
3	Dra. Sofia Aguilar	44000.0
5	Dra. Maria Castillo	43000.0
1	Dra. Ana Torres	42000.0
9	Dra. Paola Herrera	41000.0

S4. AND: estudiantes de semestre 5 o superior con promedio mayor a 8.5

Algebra relacional:

$\sigma_{\text{semestre} \geq 5 \wedge \text{promedio} > 8.5}(\text{ESTUDIANTE})$

SQL equivalente:

SELECT id_estudiante, nombre, semestre, promedio FROM estudiante WHERE semestre >= 5 AND promedio > 8.5 ORDER BY promedio DESC;

Resultado ejemplo:

id_estudiante	nombre	semestre	promedio
4	Andrea Gomez	7	9.4
12	Regina Ortega	5	9.2
1	Julio Milan	5	9.1
10	Valentina Castro	7	8.9
2	Fernanda Lopez	6	8.8
6	Natalia Perez	6	8.7
3	Diego Martinez	5	8.6

S5. OR: aulas teoricas o auditorios

Algebra relacional:

$\sigma_{\text{tipo} = \text{'Teorica'} \vee \text{tipo} = \text{'Auditorio'}}(\text{AULA})$

SQL equivalente:

SELECT id_aula, clave, tipo, capacidad FROM aula WHERE tipo = 'Teorica' OR tipo = 'Auditorio' ORDER BY id_aula;

Resultado ejemplo:

id_aula	clave	tipo	capacidad
3	B201	Teorica	45
4	B202	Teorica	50
9	E501	Auditorio	60

S6. NOT: cursos que no pertenecen al area de Matematicas

Algebra relacional:

$\sigma_{\neg}(\text{area} = \text{'Matematicas'}) (\text{CURSO})$

SQL equivalente:

SELECT id_curso, nombre, area FROM curso WHERE NOT (area = 'Matematicas') ORDER BY id_curso;

Resultado ejemplo:

id_curso	nombre	area
1	Bases de Datos	Base de Datos
2	Algoritmos y Estructuras	Programacion
5	Redes de Computadoras	Redes
6	Sistemas Operativos	Sistemas
7	Inteligencia Artificial	IA
8	Aprendizaje Automatico	IA
9	Seguridad Web	Seguridad
10	Mineria de Datos	Base de Datos
... 5 filas mas		

S7. Condicion compuesta con parentesis

Algebra relacional:

$\sigma_{(\text{semestre} \geq 6 \wedge \text{promedio} \geq 8.5) \vee (\text{edad} < 19 \wedge \text{promedio} \geq 8)} (\text{ESTUDIANTE})$

SQL equivalente:

SELECT id_estudiante, nombre, edad, semestre, promedio FROM estudiante WHERE (semestre >= 6 AND promedio >= 8.5) OR (edad < 19 AND promedio >= 8)
ORDER BY id_estudiante;

Resultado ejemplo:

id_estudiante	nombre	edad	semestre	promedio
2	Fernanda Lopez	21	6	8.8
4	Andrea Gomez	22	7	9.4
6	Natalia Perez	21	6	8.7
10	Valentina Castro	22	7	8.9
14	Ximena Vargas	18	2	8.4
20	Mariana Nunez	18	1	8.6

S8. LIKE: estudiantes cuyo nombre empieza con M o contiene ana en el correo

Algebra relacional:

$\sigma_{\text{nombre LIKE 'M\%'} \vee \text{correo LIKE '\%ana\%'}} (\text{ESTUDIANTE})$

SQL equivalente:

```
SELECT id_estudiante, nombre, correo FROM estudiante WHERE nombre LIKE 'M%' OR correo LIKE '%ana%' ORDER BY id_estudiante;
```

Resultado ejemplo:

id_estudiante	nombre	correo
9	Mateo Hernandez	mateo.hernandez@alumno.ipn.mx
20	Mariana Nunez	mariana.nunez@alumno.ipn.mx

S9. Valores NULL: estudiantes sin tutor asignado

Algebra relacional:

```
 $\sigma_{id\_tutor \text{ IS NULL}}(ESTUDIANTE)$ 
```

SQL equivalente:

```
SELECT id_estudiante, nombre, id_tutor FROM estudiante WHERE id_tutor IS NULL ORDER BY id_estudiante;
```

Resultado ejemplo:

id_estudiante	nombre	id_tutor
13	Leonardo Reyes	NULL
14	Ximena Vargas	NULL
20	Mariana Nunez	NULL

S10. Rangos BETWEEN: estudiantes de 20 a 22 años

Algebra relacional:

```
 $\sigma_{edad \text{ BETWEEN } 20 \text{ AND } 22}(ESTUDIANTE)$ 
```

SQL equivalente:

```
SELECT id_estudiante, nombre, edad FROM estudiante WHERE edad BETWEEN 20 AND 22 ORDER BY edad, nombre;
```

Resultado ejemplo:

id_estudiante	nombre	edad
8	Camila Sanchez	20
3	Diego Martinez	20
1	Julio Milan	20
12	Regina Ortega	20
16	Renata Rojas	20
2	Fernanda Lopez	21
13	Leonardo Reyes	21
18	Lucia Fuentes	21
... 4 filas mas		

2.3 Ejercicio 1.3 - Operador de proyeccion (π)

P1. Proyeccion de un solo atributo: nombres de carreras

Algebra relacional:

```
 $\pi_{nombre}(CARRERA)$ 
```

SQL equivalente:


```
SELECT DISTINCT nombre FROM carrera ORDER BY nombre;
```

Resultado ejemplo:

nombre
Ciencia de Datos
Ingenieria de Software
Ingenieria en Inteligencia Artificial
Ingenieria en Sistemas Computacionales
Seguridad Informatica

P2. Proyeccion de multiples atributos: nombre y correo de estudiantes

Algebra relacional:

$$\pi_{\text{nombre}, \text{correo}}(\text{ESTUDIANTE})$$

SQL equivalente:

```
SELECT DISTINCT nombre, correo FROM estudiante ORDER BY nombre;
```

Resultado ejemplo:

nombre	correo
Andrea Gomez	andrea.gomez@alumno.ipn.mx
Bruno Salazar	bruno.salazar@alumno.ipn.mx
Camila Sanchez	camila.sanchez@alumno.ipn.mx
Diego Martinez	diego.martinez@alumno.ipn.mx
Emiliano Ruiz	emiliano.ruiz@alumno.ipn.mx
Fernanda Lopez	fernanda.lopez@alumno.ipn.mx
Ivan Campos	ivan.campos@alumno.ipn.mx
Julio Milan	julio.milan@alumno.ipn.mx
... 12 filas mas	

P3. Proyeccion con renombramiento

Algebra relacional:

$$\rho(\text{nombre_estudiante} \leftarrow \text{nombre}, \text{email} \leftarrow \text{correo})(\pi_{\text{nombre}, \text{correo}}(\text{ESTUDIANTE}))$$

SQL equivalente:

```
SELECT DISTINCT nombre AS nombre_estudiante, correo AS email FROM estudiante ORDER BY nombre_estudiante;
```

Resultado ejemplo:

nombre_estudiante	email
Andrea Gomez	andrea.gomez@alumno.ipn.mx
Bruno Salazar	bruno.salazar@alumno.ipn.mx
Camila Sanchez	camila.sanchez@alumno.ipn.mx
Diego Martinez	diego.martinez@alumno.ipn.mx
Emiliano Ruiz	emiliano.ruiz@alumno.ipn.mx
Fernanda Lopez	fernanda.lopez@alumno.ipn.mx
Ivan Campos	ivan.campos@alumno.ipn.mx
Julio Milan	julio.milan@alumno.ipn.mx

nombre_estudiante	email
... 12 filas mas	

P4. Proyeccion que genera duplicados y elimina repetidos: semestres existentes

Algebra relacional:

$\pi_{\text{semestre}}(\text{ESTUDIANTE})$

SQL equivalente:

SELECT DISTINCT semestre FROM estudiante ORDER BY semestre;

Resultado ejemplo:

semestre
1
2
3
4
5
6
7
8

P5. Proyeccion con atributo calculado: promedio ajustado

Algebra relacional:

$\pi_{\text{nombre}, \text{promedio}, (\text{promedio} + 0.10 \rightarrow \text{promedio_estimado})}(\text{ESTUDIANTE})$

SQL equivalente:

SELECT nombre, promedio, ROUND(promedio + 0.10, 2) AS promedio_estimado FROM estudiante ORDER BY nombre LIMIT 10;

Resultado ejemplo:

nombre	promedio	promedio_estimado
Andrea Gomez	9.4	9.5
Bruno Salazar	6.9	7.0
Camila Sanchez	9.0	9.1
Diego Martinez	8.6	8.7
Emiliano Ruiz	7.9	8.0
Fernanda Lopez	8.8	8.9
Ivan Campos	7.2	7.3
Julio Milan	9.1	9.2
... 2 filas mas		

P6. Proyeccion de clave primaria

Algebra relacional:

$\pi_{\text{id_estudiante}}(\text{ESTUDIANTE})$

SQL equivalente:

SELECT DISTINCT id_estudiante FROM estudiante ORDER BY id_estudiante;

Resultado ejemplo:

id_estudiante
1
2
3
4
5
6
7
8
... 12 filas mas

P7. Todos los atributos excepto tutor: vista academica del estudiante

Algebra relacional:

$\pi_{id_estudiante, nombre, correo, edad, semestre, promedio, id_carrera}(ESTUDIANTE)$

SQL equivalente:

SELECT id_estudiante, nombre, correo, edad, semestre, promedio, id_carrera FROM estudiante ORDER BY id_estudiante LIMIT 10;

Resultado ejemplo:

id_estudiante	nombre	correo	edad	semestre	promedio	id_carrera
1	Julio Milan	julio.milan@alumno.ipn.mx	20	5	9.1	1
2	Fernanda Lopez	fernanda.lopez@alumno.ipn.mx	21	6	8.8	1
3	Diego Martinez	diego.martinez@alumno.ipn.mx	20	5	8.6	1
4	Andrea Gomez	andrea.gomez@alumno.ipn.mx	22	7	9.4	2
5	Emiliano Ruiz	emiliano.ruiz@alumno.ipn.mx	19	3	7.9	3
6	Natalia Perez	natalia.perez@alumno.ipn.mx	21	6	8.7	3
7	Santiago Flores	santiago.flores@alumno.ipn.mx	23	8	8.1	4
8	Camila Sanchez	camila.sanchez@alumno.ipn.mx	20	4	9.0	4
... 2 filas mas						

P8. Proyeccion combinada con seleccion

Algebra relacional:

$\pi_{nombre, promedio}(\sigma_{promedio \geq 9}(ESTUDIANTE))$

SQL equivalente:

SELECT DISTINCT nombre, promedio FROM estudiante WHERE promedio >= 9 ORDER BY promedio DESC;

Resultado ejemplo:

nombre	promedio
Renata Rojas	9.5
Andrea Gomez	9.4
Regina Ortega	9.2
Julio Milan	9.1
Camila Sanchez	9.0

P9. Atributos de diferentes tipos de datos

Algebra relacional:

$\pi_{\text{nombre, edad, promedio, correo}}(\text{ESTUDIANTE})$

SQL equivalente:

SELECT DISTINCT nombre, edad, promedio, correo FROM estudiante ORDER BY nombre LIMIT 10;

Resultado ejemplo:

nombre	edad	promedio	correo
Andrea Gomez	22	9.4	andrea.gomez@alumno.ipn.mx
Bruno Salazar	23	6.9	bruno.salazar@alumno.ipn.mx
Camila Sanchez	20	9.0	camila.sanchez@alumno.ipn.mx
Diego Martinez	20	8.6	diego.martinez@alumno.ipn.mx
Emiliano Ruiz	19	7.9	emiliano.ruiz@alumno.ipn.mx
Fernanda Lopez	21	8.8	fernanda.lopez@alumno.ipn.mx
Ivan Campos	19	7.2	ivan.campos@alumno.ipn.mx
Julio Milan	20	9.1	julio.milan@alumno.ipn.mx
... 2 filas mas			

P10. Proyeccion que resulta en relacion vacia

Algebra relacional:

$\pi_{\text{nombre}}(\sigma_{\text{promedio} > 10}(\text{ESTUDIANTE}))$

SQL equivalente:

SELECT DISTINCT nombre FROM estudiante WHERE promedio > 10;

Resultado ejemplo:

nombre

2.4 Ejercicio 1.4 - Operadores de teoria de conjuntos

U1. Union simple compatible: personas academicas con nombre y correo

Algebra relacional:

$\pi_{\text{nombre, correo}}(\text{ESTUDIANTE}) \cup \pi_{\text{nombre, correo}}(\text{PROFESOR})$

SQL equivalente:

SELECT nombre, correo FROM estudiante UNION SELECT nombre, correo FROM profesor ORDER BY nombre;

Resultado ejemplo:

nombre	correo
Andrea Gomez	andrea.gomez@alumno.ipn.mx
Bruno Salazar	bruno.salazar@alumno.ipn.mx
Camila Sanchez	camila.sanchez@alumno.ipn.mx
Diego Martinez	diego.martinez@alumno.ipn.mx
Dra. Ana Torres	ana.torres@ipn.mx
Dra. Elena Navarro	elena.navarro@ipn.mx
Dra. Maria Castillo	maria.castillo@ipn.mx
Dra. Paola Herrera	paola.herrera@ipn.mx
Dra. Sofia Aguilar	sofia.aguilar@ipn.mx
Dra. Valeria Soto	valeria.soto@ipn.mx
Emiliano Ruiz	emiliano.ruiz@alumno.ipn.mx
Fernanda Lopez	fernanda.lopez@alumno.ipn.mx
... 20 filas mas	

U2. Union de selecciones: estudiantes destacados o de primer ciclo

Algebra relacional:

$$\pi_{id_estudiante, nombre}(\sigma_{promedio \geq 9}(ESTUDIANTE)) \cup \pi_{id_estudiante, nombre}(\sigma_{semestre \leq 2}(ESTUDIANTE))$$

SQL equivalente:

```
SELECT id_estudiante, nombre FROM estudiante WHERE promedio >= 9 UNION SELECT id_estudiante, nombre FROM estudiante WHERE semestre <= 2 ORDER BY id_estudiante;
```

Resultado ejemplo:

id_estudiante	nombre
1	Julio Milan
4	Andrea Gomez
8	Camila Sanchez
9	Mateo Hernandez
12	Regina Ortega
14	Ximena Vargas
16	Renata Rojas
20	Mariana Nunez

U3. Union de proyecciones: areas de cursos y nombres de departamentos

Algebra relacional:

$$\pi_{area}(CURSO) \cup \rho(area \leftarrow nombre)(\pi_{nombre}(DEPARTAMENTO))$$

SQL equivalente:

```
SELECT area AS concepto FROM curso UNION SELECT nombre AS concepto FROM departamento ORDER BY concepto;
```

Resultado ejemplo:

concepto
Base de Datos
Computacion

concepto
Gestion
Gestion Academica
Hardware
Humanidades
IA
Inteligencia Artificial
... 5 filas mas

U4. Union con relacion vacia

Algebra relacional:

$$\pi_{\text{nombre}}(\text{ESTUDIANTE}) \cup \pi_{\text{nombre}}(\sigma_{\text{promedio} > 10}(\text{ESTUDIANTE}))$$

SQL equivalente:

```
SELECT nombre FROM estudiante UNION SELECT nombre FROM estudiante WHERE promedio > 10 ORDER BY nombre;
```

Resultado ejemplo:

nombre
Andrea Gomez
Bruno Salazar
Camila Sanchez
Diego Martinez
Emiliano Ruiz
Fernanda Lopez
Ivan Campos
Julio Milan
Leonardo Reyes
Lucia Fuentes
... 10 filas mas

I1. Interseccion simple: nombres que aparecen como estudiante y profesor

Algebra relacional:

$$\pi_{\text{nombre}}(\text{ESTUDIANTE}) \cap \pi_{\text{nombre}}(\text{PROFESOR})$$

SQL equivalente:

```
SELECT nombre FROM estudiante INTERSECT SELECT nombre FROM profesor ORDER BY nombre;
```

Resultado ejemplo:

nombre

I2. Interseccion con condiciones: estudiantes ISC y promedio alto

Algebra relacional:

$$\pi_{\text{id_estudiante}}(\sigma_{\text{id_carrera}=1}(\text{ESTUDIANTE})) \cap \pi_{\text{id_estudiante}}(\sigma_{\text{promedio} \geq 8.5}(\text{ESTUDIANTE}))$$

SQL equivalente:

```
SELECT id_estudiante FROM estudiante WHERE id_carrera=1 INTERSECT SELECT id_estudiante FROM estudiante WHERE promedio>=8.5 ORDER BY id_estudiante;
```

Resultado ejemplo:

id_estudiante
1
2
3

I3. Interseccion vacia: cursos obligatorios y optativos

Algebra relacional:

$$\pi_{id_curso}(\sigma_{tipo='Obligatorio'}(CURSO)) \cap \pi_{id_curso}(\sigma_{tipo='Optativo'}(CURSO))$$

SQL equivalente:

```
SELECT id_curso FROM curso WHERE tipo='Obligatorio' INTERSECT SELECT id_curso FROM curso WHERE tipo='Optativo';
```

Resultado ejemplo:

id_curso

I4. Interseccion de proyecciones: departamentos con carreras y profesores

Algebra relacional:

$$\pi_{id_departamento}(CARRERA) \cap \pi_{id_departamento}(PROFESOR)$$

SQL equivalente:

```
SELECT id_departamento FROM carrera INTERSECT SELECT id_departamento FROM profesor ORDER BY id_departamento;
```

Resultado ejemplo:

id_departamento
1
3
4

D1. Diferencia simple: profesores sin supervisor

Algebra relacional:

$$\pi_{id_profesor}(PROFESOR) - \pi_{id_profesor}(\sigma_{id_supervisor \text{ IS NOT NULL}}(PROFESOR))$$

SQL equivalente:

```
SELECT id_profesor FROM profesor EXCEPT SELECT id_profesor FROM profesor WHERE id_supervisor IS NOT NULL ORDER BY id_profesor;
```

Resultado ejemplo:

id_profesor
1
3
5

D2. Diferencia de selecciones: estudiantes no destacados

Algebra relacional:

$$\pi_{id_estudiante,nombre}(ESTUDIANTE) - \pi_{id_estudiante,nombre}(\sigma_{promedio \geq 9}(ESTUDIANTE))$$

SQL equivalente:

```
SELECT id_estudiante,nombre FROM estudiante EXCEPT SELECT id_estudiante,nombre FROM estudiante WHERE promedio>=9 ORDER BY id_estudiante;
```

Resultado ejemplo:

id_estudiante	nombre
2	Fernanda Lopez
3	Diego Martinez
5	Emiliano Ruiz
6	Natalia Perez
7	Santiago Flores
9	Mateo Hernandez
10	Valentina Castro
11	Sebastian Morales
13	Leonardo Reyes
14	Ximena Vargas
15	Pablo Medina
17	Bruno Salazar
... 3 filas mas	

D3. Diferencia que elimina cursos con grupo

Algebra relacional:

$$\pi_{id_curso}(CURSO) - \pi_{id_curso}(GRUPO)$$

SQL equivalente:

```
SELECT id_curso FROM curso EXCEPT SELECT id_curso FROM grupo ORDER BY id_curso;
```

Resultado ejemplo:

id_curso

D4. Diferencia con relacion vacia

Algebra relacional:

$$\pi_{nombre}(CARRERA) - \pi_{nombre}(\sigma_{creditos_totales>400}(CARRERA))$$

SQL equivalente:

```
SELECT nombre FROM carrera EXCEPT SELECT nombre FROM carrera WHERE creditos_totales>400 ORDER BY nombre;
```

Resultado ejemplo:

nombre
Ciencia de Datos
Ingenieria de Software
Ingenieria en Inteligencia Artificial
Ingenieria en Sistemas Computacionales
Seguridad Informatica

2.5 Ejercicio 1.5 - Producto cartesiano (x)

X1. Producto cartesiano simple de departamentos y carreras

Algebra relacional:

DEPARTAMENTO x CARRERA

SQL equivalente:

SELECT d.nombre AS departamento, c.nombre AS carrera FROM departamento d CROSS JOIN carrera c ORDER BY departamento, carrera LIMIT 12;

Explicacion: Cardinalidad completa: 5 departamentos x 5 carreras = 25 tuplas. Se muestran 12.

Resultado ejemplo:

departamento	carrera
Computacion	Ciencia de Datos
Computacion	Ingenieria de Software
Computacion	Ingenieria en Inteligencia Artificial
Computacion	Ingenieria en Sistemas Computacionales
Computacion	Seguridad Informatica
Gestion Academica	Ciencia de Datos
Gestion Academica	Ingenieria de Software
Gestion Academica	Ingenieria en Inteligencia Artificial
... 4 filas mas	

X2. Producto cartesiano seguido de seleccion simulando JOIN

Algebra relacional:

$\sigma_{\text{DEPARTAMENTO.id_departamento}=\text{CARRERA.id_departamento}}(\text{DEPARTAMENTO} \times \text{CARRERA})$

SQL equivalente:

SELECT d.nombre AS departamento, c.nombre AS carrera FROM departamento d CROSS JOIN carrera c WHERE d.id_departamento = c.id_departamento
ORDER BY
departamento;

Explicacion: Genera 25 combinaciones iniciales y conserva solo las que coinciden por id_departamento.

Resultado ejemplo:

departamento	carrera
Computacion	Ingenieria en Sistemas Computacionales
Computacion	Seguridad Informatica
Inteligencia Artificial	Ciencia de Datos
Inteligencia Artificial	Ingenieria en Inteligencia Artificial
Sistemas	Ingenieria de Software

X3. Producto cartesiano de tres relaciones pequenas

Algebra relacional:

DEPARTAMENTO x CARRERA x AULA

SQL equivalente:

SELECT d.nombre AS departamento, c.nombre AS carrera, a.clave AS aula FROM departamento d CROSS JOIN carrera c CROSS JOIN aula a LIMIT 12;

Explicacion: Cardinalidad completa: 5 x 5 x 10 = 250 tuplas. Se muestran 12.

Resultado ejemplo:

departamento	carrera	aula
Computacion	Ingenieria en Sistemas Computacionales	A101
Computacion	Ingenieria en Sistemas Computacionales	A102
Computacion	Ingenieria en Sistemas Computacionales	B201
Computacion	Ingenieria en Sistemas Computacionales	B202
Computacion	Ingenieria en Sistemas Computacionales	C301
Computacion	Ingenieria en Sistemas Computacionales	C302
Computacion	Ingenieria en Sistemas Computacionales	D401
Computacion	Ingenieria en Sistemas Computacionales	D402
... 4 filas mas		

X4. Producto cartesiano seguido de proyeccion

Algebra relacional:

$\pi_{\text{nombre_estudiante, nombre_curso}}(\text{ESTUDIANTE} \times \text{CURSO})$

SQL equivalente:

SELECT e.nombre AS estudiante, c.nombre AS curso FROM estudiante e CROSS JOIN curso c LIMIT 12;

Explicacion: Cardinalidad completa: 20 x 15 = 300 tuplas. Se muestran 12.

Resultado ejemplo:

estudiante	curso
Julio Milan	Bases de Datos
Julio Milan	Algoritmos y Estructuras
Julio Milan	Calculo Vectorial
Julio Milan	Algebra Lineal
Julio Milan	Redes de Computadoras
Julio Milan	Sistemas Operativos
Julio Milan	Inteligencia Artificial
Julio Milan	Aprendizaje Automatico
... 4 filas mas	

X5. Producto cartesiano con renombramiento para evitar ambigüedad

Algebra relacional:

$\rho_{E1}(\text{ESTUDIANTE}) \times \rho_{E2}(\text{ESTUDIANTE})$

SQL equivalente:

SELECT e1.nombre AS estudiante_1, e2.nombre AS estudiante_2 FROM estudiante e1 CROSS JOIN estudiante e2 WHERE e1.id_estudiante < e2.id_estudiante
LIMIT 12;

Explicacion: Autoproducto con alias. Sin filtro: 20 x 20 = 400; con id1 < id2 quedan pares no repetidos.

Resultado ejemplo:

estudiante_1	estudiante_2
Julio Milan	Fernanda Lopez
Julio Milan	Diego Martinez

estudiante_1	estudiante_2
Julio Milan	Andrea Gomez
Julio Milan	Emiliano Ruiz
Julio Milan	Natalia Perez
Julio Milan	Santiago Flores
Julio Milan	Camila Sanchez
Julio Milan	Mateo Hernandez
... 4 filas mas	

3. Parte 2 - Operadores avanzados del Algebra Relacional

3.1 Ejercicio 2.1 - Operadores de reunion (JOIN)

JN1. Reunion natural simple: carrera con departamento

Algebra relacional:

```
CARRERA JOIN DEPARTAMENTO
```

SQL equivalente:

```
SELECT c.nombre AS carrera, d.nombre AS departamento FROM carrera c JOIN departamento d USING(id_departamento) ORDER BY carrera;
```

Resultado ejemplo:

carrera	departamento
Ciencia de Datos	Inteligencia Artificial
Ingenieria de Software	Sistemas
Ingenieria en Inteligencia Artificial	Inteligencia Artificial
Ingenieria en Sistemas Computacionales	Computacion
Seguridad Informatica	Computacion

JN2. Reunion natural de tres tablas: estudiante, carrera y departamento

Algebra relacional:

```
ESTUDIANTE JOIN CARRERA JOIN DEPARTAMENTO
```

SQL equivalente:

```
SELECT e.nombre AS estudiante, c.nombre AS carrera, d.nombre AS departamento FROM estudiante e JOIN carrera c USING(id_carrera) JOIN departamento d ON c.id_departamento=d.id_departamento ORDER BY estudiante LIMIT 12;
```

Resultado ejemplo:

estudiante	carrera	departamento
Andrea Gomez	Ingenieria en Inteligencia Artificial	Inteligencia Artificial
Bruno Salazar	Ingenieria en Sistemas Computacionales	Computacion
Camila Sanchez	Ingenieria de Software	Sistemas
Diego Martinez	Ingenieria en Sistemas Computacionales	Computacion
Emiliano Ruiz	Ciencia de Datos	Inteligencia Artificial
Fernanda Lopez	Ingenieria en Sistemas Computacionales	Computacion
Ivan Campos	Ingenieria de Software	Sistemas
Julio Milan	Ingenieria en Sistemas Computacionales	Computacion
... 4 filas mas		

JN3. Reunion natural con proyeccion

Algebra relacional:

```
 $\pi_{\text{nombre\_estudiante, nombre\_carrera}}(\text{ESTUDIANTE JOIN CARRERA})$ 
```

SQL equivalente:

```
SELECT e.nombre AS estudiante, c.nombre AS carrera FROM estudiante e JOIN carrera c USING(id_carrera) ORDER BY e.nombre LIMIT 12;
```

Resultado ejemplo:

estudiante	carrera
Andrea Gomez	Ingenieria en Inteligencia Artificial
Bruno Salazar	Ingenieria en Sistemas Computacionales
Camila Sanchez	Ingenieria de Software
Diego Martinez	Ingenieria en Sistemas Computacionales
Emiliano Ruiz	Ciencia de Datos
Fernanda Lopez	Ingenieria en Sistemas Computacionales
Ivan Campos	Ingenieria de Software
Julio Milan	Ingenieria en Sistemas Computacionales
... 4 filas mas	

JT1. Theta join de igualdad: grupos con curso correspondiente

Algebra relacional:

```
GRUPO JOIN _GRUPO.id_curso=CURSO.id_curso CURSO
```

SQL equivalente:

```
SELECT g.id_grupo, c.nombre AS curso, g.periodo FROM grupo g JOIN curso c ON g.id_curso=c.id_curso ORDER BY g.id_grupo LIMIT 12;
```

Resultado ejemplo:

id_grupo	curso	periodo
1	Bases de Datos	2026-2
2	Algoritmos y Estructuras	2026-2
3	Calculo Vectorial	2026-2
4	Algebra Lineal	2026-2
5	Redes de Computadoras	2026-2
6	Sistemas Operativos	2026-2
7	Inteligencia Artificial	2026-2
8	Aprendizaje Automatico	2026-2
... 4 filas mas		

JT2. Theta join con desigualdad: aulas con capacidad mayor que cupo del grupo

Algebra relacional:

```
AULA JOIN _capacidad>cupo GRUPO
```

SQL equivalente:

```
SELECT a.clave, a.capacidad, g.id_grupo, g.cupo FROM aula a JOIN grupo g ON a.capacidad > g.cupo ORDER BY a.id_aula, g.id_grupo LIMIT 12;
```

Resultado ejemplo:

clave	capacidad	id_grupo	cupo
A101	35	5	30
A101	35	6	30
A101	35	7	25
A101	35	8	25
A101	35	13	30
A101	35	14	30

clave	capacidad	id_grupo	cupo
A101	35	15	28
A101	35	19	30
... 4 filas mas			

JT3. Theta join compuesto: grupo en aula laboratorio con cupo suficiente

Algebra relacional:

```
AULA JOIN _(id_aula=id_aula ^ capacidad≥cupo ^ tipo LIKE 'Laboratorio%') GRUPO
```

SQL equivalente:

```
SELECT g.id_grupo, a.clave, a.tipo, a.capacidad, g.cupo FROM grupo g JOIN aula a ON g.id_aula=a.id_aula AND a.capacidad >= g.cupo AND a.tipo LIKE 'Laboratorio%' ORDER BY g.id_grupo;
```

Resultado ejemplo:

id_grupo	clave	tipo	capacidad	cupo
1	A101	Laboratorio	35	35
2	A102	Laboratorio	40	35
5	C301	Laboratorio	30	30
6	C302	Laboratorio	32	30
7	D401	Laboratorio IA	25	25
8	D402	Laboratorio IA	30	25
9	A101	Laboratorio	35	35
10	A102	Laboratorio	40	35
... 8 filas mas				

J01. Left outer join: todos los profesores y sus grupos si existen

Algebra relacional:

```
PROFESOR LEFT OUTER JOIN GRUPO
```

SQL equivalente:

```
SELECT p.nombre AS profesor, g.id_grupo, g.periodo FROM profesor p LEFT JOIN grupo g ON p.id_profesor=g.id_profesor ORDER BY p.id_profesor, g.id_grupo LIMIT 12;
```

Resultado ejemplo:

profesor	id_grupo	periodo
Dra. Ana Torres	1	2026-2
Dra. Ana Torres	2	2026-2
Dra. Ana Torres	12	2026-2
Dra. Ana Torres	18	2026-1
Mtro. Luis Ramirez	16	2026-1
Dra. Sofia Aguilar	7	2026-2
Dra. Sofia Aguilar	10	2026-2
Mtro. Carlos Vega	3	2026-2
... 4 filas mas		

J02. Right outer join: todos los grupos con profesor, equivalente mostrado como left desde grupo

Algebra relacional:

```
PROFESOR RIGHT OUTER JOIN GRUPO
```

SQL equivalente:

```
SELECT g.id_grupo, c.nombre AS curso, p.nombre AS profesor FROM grupo g LEFT JOIN profesor p ON p.id_profesor=g.id_profesor JOIN curso c ON c.id_curso=g.id_curso ORDER BY g.id_grupo LIMIT 12;
```

Resultado ejemplo:

id_grupo	curso	profesor
1	Bases de Datos	Dra. Ana Torres
2	Algoritmos y Estructuras	Dra. Ana Torres
3	Calculo Vectorial	Mtro. Carlos Vega
4	Algebra Lineal	Dra. Paola Herrera
5	Redes de Computadoras	Dra. Maria Castillo
6	Sistemas Operativos	Dra. Maria Castillo
7	Inteligencia Artificial	Dra. Sofia Aguilar
8	Aprendizaje Automatico	Dra. Elena Navarro
... 4 filas mas		

J03. Full outer join: cursos y grupos, conservando cursos sin grupo y grupos sin curso

Algebra relacional:

```
CURSO FULL OUTER JOIN GRUPO
```

SQL equivalente:

```
SELECT c.nombre AS curso, g.id_grupo, g.periodo FROM curso c LEFT JOIN grupo g ON c.id_curso=g.id_curso UNION SELECT c.nombre AS curso, g.id_grupo, g.periodo FROM grupo g LEFT JOIN curso c ON c.id_curso=g.id_curso ORDER BY curso, id_grupo LIMIT 12;
```

Resultado ejemplo:

curso	id_grupo	periodo
Administracion de Proyectos	11	2026-2
Algebra Lineal	4	2026-2
Algoritmos y Estructuras	2	2026-2
Algoritmos y Estructuras	17	2026-1
Aprendizaje Automatico	8	2026-2
Aprendizaje Automatico	20	2026-2
Arquitectura de Computadoras	13	2026-2
Bases de Datos	1	2026-2
... 4 filas mas		

JS1. Semi-join: estudiantes que tienen inscripciones

Algebra relacional:

```
ESTUDIANTE SEMIJOIN INSCRIPCION
```

SQL equivalente:

```
SELECT DISTINCT e.id_estudiante, e.nombre FROM estudiante e WHERE EXISTS (SELECT 1 FROM inscripcion i WHERE i.id_estudiante=e.id_estudiante)
ORDER BY
e.id_estudiante;
```

Resultado ejemplo:

id_estudiante	nombre
1	Julio Milan
2	Fernanda Lopez
3	Diego Martinez
4	Andrea Gomez
5	Emiliano Ruiz
6	Natalia Perez
7	Santiago Flores
8	Camila Sanchez
9	Mateo Hernandez
10	Valentina Castro
11	Sebastian Morales
12	Regina Ortega
... 8 filas mas	

JA1. Anti-join: estudiantes sin inscripciones

Algebra relacional:

ESTUDIANTE ANTIJOIN INSCRIPCION

SQL equivalente:

```
SELECT e.id_estudiante, e.nombre FROM estudiante e WHERE NOT EXISTS (SELECT 1 FROM inscripcion i WHERE i.id_estudiante=e.id_estudiante) ORDER
BY
e.id_estudiante;
```

Resultado ejemplo:

id_estudiante	nombre
---------------	--------

JS2. Semi-join combinado: carreras con estudiantes inscritos

Algebra relacional:

CARRERA SEMIJOIN (ESTUDIANTE JOIN INSCRIPCION)

SQL equivalente:

```
SELECT DISTINCT c.id_carrera, c.nombre FROM carrera c WHERE EXISTS (SELECT 1 FROM estudiante e JOIN inscripcion i ON
e.id_estudiante=i.id_estudiante
WHERE e.id_carrera=c.id_carrera) ORDER BY c.id_carrera;
```

Resultado ejemplo:

id_carrera	nombre
1	Ingenieria en Sistemas Computacionales
2	Ingenieria en Inteligencia Artificial
3	Ciencia de Datos
4	Ingenieria de Software
5	Seguridad Informatica

SELF1. Auto-reunion: pares de estudiantes de la misma carrera

Algebra relacional:

$\rho_{E1}(\text{ESTUDIANTE}) \text{ JOIN } _E1.id_carrera=E2.id_carrera \wedge E1.id < E2.id \rho_{E2}(\text{ESTUDIANTE})$

SQL equivalente:

```
SELECT e1.nombre AS estudiante_1, e2.nombre AS estudiante_2, e1.id_carrera FROM estudiante e1 JOIN estudiante e2 ON
e1.id_carrera=e2.id_carrera AND
e1.id_estudiante<e2.id_estudiante ORDER BY e1.id_carrera LIMIT 12;
```

Resultado ejemplo:

estudiante_1	estudiante_2	id_carrera
Julio Milan	Bruno Salazar	1
Julio Milan	Diego Martinez	1
Julio Milan	Fernanda Lopez	1
Julio Milan	Sebastian Morales	1
Fernanda Lopez	Bruno Salazar	1
Fernanda Lopez	Diego Martinez	1
Fernanda Lopez	Sebastian Morales	1
Diego Martinez	Bruno Salazar	1
... 4 filas mas		

SELF2. Auto-reunion jerarquica: profesor y supervisor

Algebra relacional:

$\rho_P(\text{PROFESOR}) \text{ JOIN } _P.id_supervisor=S.id_profesor \rho_S(\text{PROFESOR})$

SQL equivalente:

```
SELECT p.nombre AS profesor, s.nombre AS supervisor FROM profesor p JOIN profesor s ON p.id_supervisor=s.id_profesor ORDER BY supervisor,
profesor;
```

Resultado ejemplo:

profesor	supervisor
Mtro. Carlos Vega	Dra. Ana Torres
Mtro. Jorge Pineda	Dra. Ana Torres
Mtro. Luis Ramirez	Dra. Ana Torres
Ing. Roberto Diaz	Dra. Maria Castillo
Mtro. Daniel Cruz	Dra. Maria Castillo
Dra. Elena Navarro	Dra. Sofia Aguilar
Dra. Valeria Soto	Dra. Sofia Aguilar
Dra. Paola Herrera	Mtro. Carlos Vega
... 1 filas mas	

SELF3. Auto-reunion por características similares: profesores del mismo departamento

Algebra relacional:

$\rho_{P1}(\text{PROFESOR}) \text{ JOIN } _P1.id_departamento=P2.id_departamento \wedge P1.id < P2.id \rho_{P2}(\text{PROFESOR})$

SQL equivalente:

```
SELECT p1.nombre AS profesor_1, p2.nombre AS profesor_2, p1.id_departamento FROM profesor p1 JOIN profesor p2 ON
p1.id_departamento=p2.id_departamento
AND p1.id_profesor<p2.id_profesor ORDER BY p1.id_departamento LIMIT 12;
```

Resultado ejemplo:

profesor_1	profesor_2	id_departamento
Dra. Ana Torres	Ing. Ricardo Mora	1
Dra. Ana Torres	Mtro. Jorge Pineda	1
Dra. Ana Torres	Mtro. Luis Ramirez	1
Mtro. Luis Ramirez	Ing. Ricardo Mora	1
Mtro. Luis Ramirez	Mtro. Jorge Pineda	1
Mtro. Jorge Pineda	Ing. Ricardo Mora	1
Mtro. Carlos Vega	Dra. Paola Herrera	2
Dra. Maria Castillo	Ing. Roberto Diaz	3
... 3 filas mas		

3.2 Ejercicio 2.2 - Funciones de agregacion

A1. COUNT de todas las tuplas: total de estudiantes

Algebra relacional:

```
F__COUNT(*) (ESTUDIANTE)
```

SQL equivalente:

```
SELECT COUNT(*) AS total_estudiantes FROM estudiante;
```

Resultado ejemplo:

total_estudiantes
20

A2. COUNT de valores distintos: carreras con estudiantes

Algebra relacional:

```
F__COUNT_DISTINCT(id_carrera) (ESTUDIANTE)
```

SQL equivalente:

```
SELECT COUNT(DISTINCT id_carrera) AS carreras_con_estudiantes FROM estudiante;
```

Resultado ejemplo:

carreras_con_estudiantes
5

A3. SUM de columna numerica: cupo total de grupos

Algebra relacional:

```
F__SUM(cupo) (GRUPO)
```

SQL equivalente:

```
SELECT SUM(cupo) AS cupo_total FROM grupo;
```

Resultado ejemplo:

cupo_total
663

A4. AVG de columna numerica: promedio general academico

Algebra relacional:

F__AVG(promedio)(ESTUDIANTE)

SQL equivalente:

SELECT ROUND(AVG(promedio),2) AS promedio_general FROM estudiante;

Resultado ejemplo:

promedio_general
8.4

A5. MAX de columna numerica: salario maximo docente

Algebra relacional:

F__MAX(salario)(PROFESOR)

SQL equivalente:

SELECT MAX(salario) AS salario_maximo FROM profesor;

Resultado ejemplo:

salario_maximo
46000.0

A6. MIN de columna numerica: calificacion minima

Algebra relacional:

F__MIN(calificacion)(INSCRIPCION)

SQL equivalente:

SELECT MIN(calificacion) AS calificacion_minima FROM inscripcion;

Resultado ejemplo:

calificacion_minima
5.7

A7. Multiples funciones agregadas

Algebra relacional:

F__COUNT(*),AVG(promedio),MAX(promedio),MIN(promedio)(ESTUDIANTE)

SQL equivalente:

SELECT COUNT(*) AS n, ROUND(AVG(promedio),2) AS promedio, MAX(promedio) AS maximo, MIN(promedio) AS minimo FROM estudiante;

Resultado ejemplo:

n	promedio	maximo	minimo
20	8.4	9.5	6.9

A8. Agregacion con seleccion previa

Algebra relacional:

F__AVG(promedio)($\sigma_{id_carrera=1}$ (ESTUDIANTE))

SQL equivalente:

SELECT ROUND(AVG(promedio),2) AS promedio_isc FROM estudiante WHERE id_carrera=1;

Resultado ejemplo:

promedio_isc
8.34

A9. Agregacion con condicion de resultado

Algebra relacional:

$\sigma_{total > 3}(G_{id_carrera} F_COUNT(*) \rightarrow total(ESTUDIANTE))$

SQL equivalente:

SELECT id_carrera, COUNT(*) AS total FROM estudiante GROUP BY id_carrera HAVING COUNT(*) > 3 ORDER BY id_carrera;

Resultado ejemplo:

id_carrera	total
1	5
2	4
3	4
4	4

A10. Agregacion sobre resultado de reunion

Algebra relacional:

F__AVG(calificacion)(ESTUDIANTE JOIN INSCRIPCION)

SQL equivalente:

SELECT ROUND(AVG(i.calificacion),2) AS promedio_calificaciones FROM estudiante e JOIN inscripcion i ON e.id_estudiante=i.id_estudiante;

Resultado ejemplo:

promedio_calificaciones
7.95

3.3 Ejercicio 2.3 - Agrupacion (γ)

G1. Agrupacion simple con COUNT: estudiantes por carrera

Algebra relacional:

$G_{id_carrera} F_COUNT(*) \rightarrow total(ESTUDIANTE)$

SQL equivalente:

SELECT c.nombre AS carrera, COUNT(e.id_estudiante) AS total FROM carrera c LEFT JOIN estudiante e ON c.id_carrera=e.id_carrera GROUP BY c.id_carrera,c.nombre ORDER BY c.id_carrera;

Resultado ejemplo:

carrera	total
Ingenieria en Sistemas Computacionales	5
Ingenieria en Inteligencia Artificial	4
Ciencia de Datos	4

carrera	total
Ingenieria de Software	4
Seguridad Informatica	3

G2. Agrupacion con SUM: creditos por area

Algebra relacional:

$G_area \ F_SUM(creditos) \rightarrow credits_area(CURSO)$

SQL equivalente:

`SELECT area, SUM(creditos) AS credits_area FROM curso GROUP BY area ORDER BY area;`

Resultado ejemplo:

area	credits_area
Base de Datos	14
Gestion	5
Hardware	7
Humanidades	4
IA	16
Matematicas	14
Programacion	14
Redes	8
... 2 filas mas	

G3. Agrupacion con AVG: promedio por carrera

Algebra relacional:

$G_id_carrera \ F_AVG(promedio) \rightarrow promedio_carrera(ESTUDIANTE)$

SQL equivalente:

`SELECT c.nombre AS carrera, ROUND(AVG(e.promedio),2) AS promedio_carrera FROM estudiante e JOIN carrera c ON e.id_carrera=c.id_carrera GROUP BY c.nombre ORDER BY promedio_carrera DESC;`

Resultado ejemplo:

carrera	promedio_carrera
Ingenieria en Inteligencia Artificial	9.18
Ingenieria en Sistemas Computacionales	8.34
Ingenieria de Software	8.18
Ciencia de Datos	8.15
Seguridad Informatica	8.13

G4. Agrupacion con MAX/MIN: calificaciones por grupo

Algebra relacional:

$G_id_grupo \ F_MAX(calificacion), MIN(calificacion)(INSCRIPCION)$

SQL equivalente:

`SELECT id_grupo, MAX(calificacion) AS max_calif, MIN(calificacion) AS min_calif FROM inscripcion GROUP BY id_grupo ORDER BY id_grupo LIMIT 12;`

Resultado ejemplo:

id_grupo	max_calif	min_calif
1	9.4	6.5
2	9.0	6.5
3	8.9	5.7
4	8.9	6.8
5	8.9	7.9
6	9.1	6.5
7	9.0	8.2
8	8.5	7.9
... 4 filas mas		

G5. Agrupacion por multiples atributos: cursos por tipo y area

Algebra relacional:

$G_{\text{tipo, area}} F_{\text{COUNT}(*)} \rightarrow \text{total}(\text{CURSO})$

SQL equivalente:

`SELECT tipo, area, COUNT(*) AS total FROM curso GROUP BY tipo, area ORDER BY tipo, area;`

Resultado ejemplo:

tipo	area	total
Obligatorio	Base de Datos	1
Obligatorio	Hardware	1
Obligatorio	Humanidades	1
Obligatorio	Matematicas	2
Obligatorio	Programacion	1
Obligatorio	Redes	1
Obligatorio	Sistemas	1
Optativo	Base de Datos	1
... 4 filas mas		

G6. Agrupacion con seleccion previa: inscritos aprobados por grupo

Algebra relacional:

$G_{\text{id_grupo}} F_{\text{COUNT}(*)} \rightarrow \text{aprobados}(\sigma_{\text{estado}='Aprobado'}(\text{INSCRIPCION}))$

SQL equivalente:

`SELECT id_grupo, COUNT(*) AS aprobados FROM inscripcion WHERE estado='Aprobado' GROUP BY id_grupo ORDER BY id_grupo LIMIT 12;`

Resultado ejemplo:

id_grupo	aprobados
1	14
2	13
3	8
4	8
5	6

id_grupo	aprobados
6	6
7	6
8	5
... 4 filas mas	

G7. Agrupacion con condicion posterior HAVING

Algebra relacional:

$$\sigma_{total > 5}(G_id_grupo \ F_COUNT(*) \rightarrow total(INSCRIPCION))$$

SQL equivalente:

```
SELECT id_grupo, COUNT(*) AS total FROM inscripcion GROUP BY id_grupo HAVING COUNT(*) > 5 ORDER BY total DESC, id_grupo;
```

Resultado ejemplo:

id_grupo	total
1	14
2	13
3	10
10	9
4	8
5	6
6	6
7	6
... 1 filas mas	

G8. Agrupacion sobre reunion: inscritos por curso

Algebra relacional:

$$G_CURSO.nombre \ F_COUNT(*) \rightarrow inscritos(CURSO \ JOIN \ GRUPO \ JOIN \ INSCRIPCION)$$

SQL equivalente:

```
SELECT c.nombre AS curso, COUNT(i.id_inscripcion) AS inscritos FROM curso c JOIN grupo g ON c.id_curso=g.id_curso JOIN inscripcion i ON g.id_grupo=i.id_grupo GROUP BY c.nombre ORDER BY inscritos DESC, curso LIMIT 12;
```

Resultado ejemplo:

curso	inscritos
Bases de Datos	16
Algoritmos y Estructuras	15
Calculo Vectorial	10
Mineria de Datos	9
Algebra Lineal	8
Programacion Web	7
Redes de Computadoras	7
Inteligencia Artificial	6
... 4 filas mas	

G9. Multiples funciones agrupadas: estadísticas por carrera

Algebra relacional:

G_id_carrera F__COUNT(*),AVG(promedio),MAX(promedio)(ESTUDIANTE)

SQL equivalente:

```
SELECT c.nombre AS carrera, COUNT(*) AS total, ROUND(AVG(e.promedio),2) AS promedio, MAX(e.promedio) AS maximo FROM estudiante e JOIN carrera c ON e.id_carrera=c.id_carrera GROUP BY c.nombre ORDER BY carrera;
```

Resultado ejemplo:

carrera	total	promedio	maximo
Ciencia de Datos	4	8.15	8.7
Ingenieria de Software	4	8.18	9.0
Ingenieria en Inteligencia Artificial	4	9.18	9.5
Ingenieria en Sistemas Computacionales	5	8.34	9.1
Seguridad Informatica	3	8.13	8.9

G10. Agrupacion anidada: promedio de promedios por departamento

Algebra relacional:

G_departamento F__AVG(promedio_carrera)(G_carrera F__AVG(promedio)(ESTUDIANTE JOIN CARRERA JOIN DEPARTAMENTO))

SQL equivalente:

```
SELECT departamento, ROUND(AVG(promedio_carrera),2) AS promedio_de_promedios FROM (SELECT d.nombre AS departamento, c.nombre AS carrera, AVG(e.promedio) AS promedio_carrera FROM estudiante e JOIN carrera c ON e.id_carrera=c.id_carrera JOIN departamento d ON c.id_departamento=d.id_departamento GROUP BY d.nombre, c.nombre) x GROUP BY departamento ORDER BY departamento;
```

Resultado ejemplo:

departamento	promedio_de_promedios
Computacion	8.24
Inteligencia Artificial	8.66
Sistemas	8.18

3.4 Ejercicio 2.4 - Division (DIV)

Para cada division se usa tambien la equivalencia general: $R \text{ DIV } S = \pi_X(R) - \pi_X((\pi_X(R) \times S) - R)$. En SQL se implementa con doble NOT EXISTS, que representa el cuantificador universal.

DIV1. Estudiantes inscritos en TODOS los cursos obligatorios

Algebra relacional:

$\pi_{id_estudiante,nombre}(ESTUDIANTE) \text{ JOIN } ((\pi_{id_estudiante,id_curso}(INSCRIPCION \text{ JOIN } GRUPO)) \text{ DIV } \pi_{id_curso}(\sigma_{tipo='Obligatorio'}(CURSO)))$

SQL equivalente:

```
SELECT e.id_estudiante, e.nombre FROM estudiante e WHERE NOT EXISTS (SELECT 1 FROM curso c WHERE c.tipo='Obligatorio' AND NOT EXISTS (SELECT 1 FROM inscripcion i JOIN grupo g ON i.id_grupo=g.id_grupo WHERE i.id_estudiante=e.id_estudiante AND g.id_curso=c.id_curso)) ORDER BY e.id_estudiante;
```

Resultado ejemplo:

id_estudiante	nombre
---------------	--------

DIV2. Estudiantes inscritos en TODOS los cursos del area Base de Datos

Algebra relacional:

$R(id_estudiante, id_curso) \text{ DIV } S(id_curso), S = \pi_{id_curso}(\sigma_{area='Base de Datos'}(CURSO))$

SQL equivalente:

```
SELECT e.id_estudiante, e.nombre FROM estudiante e WHERE NOT EXISTS (SELECT 1 FROM curso c WHERE c.area='Base de Datos' AND NOT EXISTS (SELECT 1 FROM inscripcion i JOIN grupo g ON i.id_grupo=g.id_grupo WHERE i.id_estudiante=e.id_estudiante AND g.id_curso=c.id_curso)) ORDER BY e.id_estudiante;
```

Resultado ejemplo:

id_estudiante	nombre
1	Julio Milan
2	Fernanda Lopez
3	Diego Martinez
4	Andrea Gomez
6	Natalia Perez
12	Regina Ortega
16	Renata Rojas
18	Lucia Fuentes

DIV3. Profesores que imparten TODOS los cursos del area Programacion

Algebra relacional:

$\pi_{id_profesor, nombre}(PROFESOR) \text{ JOIN } ((\pi_{id_profesor, id_curso}(GRUPO)) \text{ DIV } \pi_{id_curso}(\sigma_{area='Programacion'}(CURSO)))$

SQL equivalente:

```
SELECT p.id_profesor, p.nombre FROM profesor p WHERE NOT EXISTS (SELECT 1 FROM curso c WHERE c.area='Programacion' AND NOT EXISTS (SELECT 1 FROM grupo g WHERE g.id_profesor=p.id_profesor AND g.id_curso=c.id_curso)) ORDER BY p.id_profesor;
```

Resultado ejemplo:

id_profesor	nombre
1	Dra. Ana Torres

DIV4. Estudiantes inscritos en TODOS los cursos de Matematicas

Algebra relacional:

$R(id_estudiante, id_curso) \text{ DIV } \pi_{id_curso}(\sigma_{area='Matematicas'}(CURSO))$

SQL equivalente:

```
SELECT e.id_estudiante, e.nombre FROM estudiante e WHERE NOT EXISTS (SELECT 1 FROM curso c WHERE c.area='Matematicas' AND NOT EXISTS (SELECT 1 FROM inscripcion i JOIN grupo g ON i.id_grupo=g.id_grupo WHERE i.id_estudiante=e.id_estudiante AND g.id_curso=c.id_curso)) ORDER BY e.id_estudiante;
```

Resultado ejemplo:

id_estudiante	nombre
1	Julio Milan
2	Fernanda Lopez
3	Diego Martinez

id_estudiante	nombre
8	Camila Sanchez
9	Mateo Hernandez
13	Leonardo Reyes
19	Ivan Campos

DIV5. Estudiantes con inscripcion en TODOS los cursos optativos de IA

Algebra relacional:

$R(id_estudiante, id_curso) \text{ DIV } \pi_{id_curso}(\sigma_{area='IA'}(CURSO))$

SQL equivalente:

```
SELECT e.id_estudiante, e.nombre FROM estudiante e WHERE NOT EXISTS (SELECT 1 FROM curso c WHERE c.area='IA' AND NOT EXISTS (SELECT 1 FROM
inscripcion
i JOIN grupo g ON i.id_grupo=g.id_grupo WHERE i.id_estudiante=e.id_estudiante AND g.id_curso=c.id_curso)) ORDER BY e.id_estudiante;
```

Resultado ejemplo:

id_estudiante	nombre
4	Andrea Gomez
6	Natalia Perez
12	Regina Ortega
16	Renata Rojas
20	Mariana Nunez

4. Parte 3 - Calculo Relacional

4.1 Ejercicio 3.1 - Calculo Relacional de Tuplas (CRT)

CRT1. Variable libre simple

CRT:

$$\{t.nombre \mid ESTUDIANTE(t)\}$$

Equivalente en AR:

$$\pi_{nombre}(ESTUDIANTE)$$

Explicacion: Devuelve el atributo nombre de cada tupla estudiante.

CRT2. Condicion de atributo

CRT:

$$\{t \mid ESTUDIANTE(t) \wedge t.id_carrera = 1\}$$

Equivalente en AR:

$$\sigma_{id_carrera=1}(ESTUDIANTE)$$

Explicacion: Filtra estudiantes de ISC.

CRT3. Comparacion

CRT:

$$\{t.nombre \mid ESTUDIANTE(t) \wedge t.promedio > 8.5\}$$

Equivalente en AR:

$$\pi_{nombre}(\sigma_{promedio>8.5}(ESTUDIANTE))$$

Explicacion: Filtra por promedio mayor que 8.5.

CRT4. AND

CRT:

$$\{t.nombre \mid ESTUDIANTE(t) \wedge t.semestre \geq 5 \wedge t.promedio \geq 8.5\}$$

Equivalente en AR:

$$\pi_{nombre}(\sigma_{semestre \geq 5 \wedge promedio \geq 8.5}(ESTUDIANTE))$$

Explicacion: Ambas condiciones deben cumplirse.

CRT5. OR

CRT:

$$\{t.nombre \mid CURSO(t) \wedge (t.area = "IA" \vee t.area = "Seguridad") \}$$

Equivalente en AR:

$$\pi_{nombre}(\sigma_{area="IA" \vee area="Seguridad"}(CURSO))$$

Explicacion: Acepta cursos de cualquiera de las dos areas.

CRT6. Existencial

CRT:

$$\{e.nombre \mid ESTUDIANTE(e) \wedge \exists i (INSCRIPCION(i) \wedge i.id_estudiante=e.id_estudiante)\}$$

Equivalente en AR:

$\pi_{\text{nombre}}(\text{ESTUDIANTE SEMIJOIN INSCRIPCION})$

Explicacion: Existe al menos una inscripcion del estudiante.

CRT7. Universal

CRT:

$\{e.\text{nombre} \mid \text{ESTUDIANTE}(e) \wedge \forall c((\text{CURSO}(c) \wedge c.\text{tipo}=\text{"Obligatorio"}) \rightarrow \exists i \exists g(\text{INSCRIPCION}(i) \wedge \text{GRUPO}(g) \wedge i.\text{id_estudiante}=e.\text{id_estudiante} \wedge i.\text{id_grupo}=g.\text{id_grupo} \wedge g.\text{id_curso}=c.\text{id_curso}))\}$

Equivalente en AR:

Division entre inscripciones y cursos obligatorios

Explicacion: Para todo curso obligatorio existe una inscripcion del estudiante.

CRT8. Existencial con AND

CRT:

$\{p.\text{nombre} \mid \text{PROFESOR}(p) \wedge \exists g(\text{GRUPO}(g) \wedge g.\text{id_profesor}=p.\text{id_profesor} \wedge g.\text{periodo}=\text{"2026-2"}) \}$

Equivalente en AR:

$\pi_{\text{nombre}}(\text{PROFESOR SEMIJOIN } \sigma_{\text{periodo}=\text{"2026-2"}}(\text{GRUPO}))$

Explicacion: El profesor imparte al menos un grupo en 2026-2.

CRT9. Universal con implicacion

CRT:

$\{c.\text{nombre} \mid \text{CARRERA}(c) \wedge \forall e((\text{ESTUDIANTE}(e) \wedge e.\text{id_carrera}=c.\text{id_carrera}) \rightarrow e.\text{promedio} \geq 8)\}$

Equivalente en AR:

Carreras donde todos sus estudiantes tienen promedio ≥ 8

Explicacion: Todos los estudiantes asociados a la carrera cumplen la condicion.

CRT10. Cuantificadores anidados

CRT:

$\{p.\text{nombre} \mid \text{PROFESOR}(p) \wedge \exists g(\text{GRUPO}(g) \wedge g.\text{id_profesor}=p.\text{id_profesor} \wedge \forall i(\text{INSCRIPCION}(i) \wedge i.\text{id_grupo}=g.\text{id_grupo} \rightarrow i.\text{calificacion} \geq 6))\}$

Equivalente en AR:

Profesores con algun grupo donde todos aprobaron

Explicacion: Existe un grupo del profesor y todos sus inscritos tienen calificacion aprobatoria.

CRT11. Negacion

CRT:

$\{e.\text{nombre} \mid \text{ESTUDIANTE}(e) \wedge \neg \exists i(\text{INSCRIPCION}(i) \wedge i.\text{id_estudiante}=e.\text{id_estudiante})\}$

Equivalente en AR:

ESTUDIANTE ANTIJOIN INSCRIPCION

Explicacion: No existe inscripcion del estudiante.

CRT12. Join por predicado

CRT:

$\{<e.\text{nombre}, c.\text{nombre}> \mid \text{ESTUDIANTE}(e) \wedge \text{CARRERA}(c) \wedge e.\text{id_carrera}=c.\text{id_carrera}\}$

Equivalente en AR:

$\pi_{e.\text{nombre}, c.\text{nombre}}(\text{ESTUDIANTE JOIN CARRERA})$

Explicacion: Relaciona estudiantes con su carrera.

CRT13. Agregacion descrita

CRT:

$$\{ \langle c.nombre, n \rangle \mid CARRERA(c) \wedge n = COUNT(\{ e \mid ESTUDIANTE(e) \wedge e.id_carrera = c.id_carrera \}) \}$$

Equivalente en AR:

$$G_carrera \text{ COUNT}(estudiante)$$

Explicacion: Version extendida con conteo por carrera.

CRT14. Subconsulta EXISTS

CRT:

$$\{ e.nombre \mid ESTUDIANTE(e) \wedge \exists i \exists g \exists c (INSCRIPCION(i) \wedge GRUPO(g) \wedge CURSO(c) \wedge i.id_estudiante = e.id_estudiante \wedge i.id_grupo = g.id_grupo \wedge g.id_curso = c.id_curso \wedge c.area = "IA") \}$$

Equivalente en AR:

$$Estudiantes \text{ inscritos en cursos IA}$$

Explicacion: Existe una ruta estudiante-inscripcion-grupo-curso de area IA.

CRT15. Consulta compuesta

CRT:

$$\{ e.nombre \mid ESTUDIANTE(e) \wedge e.promedio \geq 8.5 \wedge \exists i (INSCRIPCION(i) \wedge i.id_estudiante = e.id_estudiante \wedge i.estado = "Aprobado") \}$$

Equivalente en AR:

$$\pi_nombre(\sigma_promedio \geq 8.5(ESTUDIANTE) \text{ SEMIJOIN } \sigma_estado = "Aprobado"(INSCRIPCION))$$

Explicacion: Promedio alto y al menos una inscripcion aprobada.

4.2 Ejercicio 3.2 - Calculo Relacional de Dominios (CRD)

CRD1. Consulta simple con variables de dominio

CRD:

$$\{ \langle n \rangle \mid \exists id \exists co \exists ed \exists se \exists pr \exists ca \exists tu \text{ ESTUDIANTE}(id, n, co, ed, se, pr, ca, tu) \}$$

Equivalente en AR:

$$\pi_nombre(ESTUDIANTE)$$

Variables y explicacion: n representa nombre del estudiante.

CRD2. Multiples variables

CRD:

$$\{ \langle n, co \rangle \mid \exists id \exists ed \exists se \exists pr \exists ca \exists tu \text{ ESTUDIANTE}(id, n, co, ed, se, pr, ca, tu) \}$$

Equivalente en AR:

$$\pi_nombre, correo(ESTUDIANTE)$$

Variables y explicacion: n y co son dominios nombre y correo.

CRD3. Condicion en variables

CRD:

$$\{ \langle n, pr \rangle \mid \exists id \exists co \exists ed \exists se \exists ca \exists tu \text{ ESTUDIANTE}(id, n, co, ed, se, pr, ca, tu) \wedge pr \geq 9 \}$$

Equivalente en AR:

$\pi_{\text{nombre, promedio}}(\sigma_{\text{promedio} \geq 9}(\text{ESTUDIANTE}))$

Variables y explicacion: pr filtra promedio.

CRD4. Cuantificador existencial

CRD:

$\{ \langle n \rangle \mid \exists id \exists co \exists ed \exists se \exists pr \exists ca \exists tu (\text{ESTUDIANTE}(id, n, co, ed, se, pr, ca, tu) \wedge \exists ii \exists g \exists f \exists cal \exists es \text{ INSCRIPCION}(ii, id, g, f, cal, es)) \}$

Equivalente en AR:

ESTUDIANTE SEMIJOIN INSCRIPCION

Variables y explicacion: Existe una inscripcion asociada al id.

CRD5. Cuantificador universal

CRD:

$\{ \langle n \rangle \mid \text{ESTUDIANTE}(id, n, co, ed, se, pr, ca, tu) \wedge \forall cid \forall cn \forall cr \forall ti \forall ar \forall dep ((\text{CURSO}(cid, cn, cr, ti, ar, dep) \wedge ti = \text{"Obligatorio"}) \rightarrow \exists ii \exists gid \exists f \exists cal \exists es \exists prof \exists au \exists per \exists cu \exists hor (\text{INSCRIPCION}(ii, id, gid, f, cal, es) \wedge \text{GRUPO}(gid, cid, prof, au, per, cu, hor))) \}$

Equivalente en AR:

Division obligatorios

Variables y explicacion: Todo curso obligatorio debe aparecer asociado al estudiante.

CRD6. Equivalente a JOIN

CRD:

$\{ \langle ne, nc \rangle \mid \text{ESTUDIANTE}(id, ne, co, ed, se, pr, ca, tu) \wedge \text{CARRERA}(ca, nc, nv, ct, dep) \}$

Equivalente en AR:

ESTUDIANTE JOIN CARRERA

Variables y explicacion: Comparte el dominio id_carrera.

CRD7. Equivalente a reunion externa

CRD:

$\{ \langle np, gid \rangle \mid \text{PROFESOR}(pid, np, co, gr, sa, dep, sup) \wedge (\text{GRUPO}(gid, cur, pid, a, per, cupo, hor) \vee \neg \exists g \exists cur \exists a \exists per \exists cupo \exists hor \text{ GRUPO}(g, cur, pid, a, per, cupo, hor)) \}$

Equivalente en AR:

PROFESOR LEFT OUTER JOIN GRUPO

Variables y explicacion: Conserva profesores aunque no tengan grupo.

CRD8. Consulta con negacion

CRD:

$\{ \langle n \rangle \mid \text{ESTUDIANTE}(id, n, co, ed, se, pr, ca, tu) \wedge \neg \exists ii \exists g \exists f \exists cal \exists es \text{ INSCRIPCION}(ii, id, g, f, cal, es) \}$

Equivalente en AR:

ESTUDIANTE ANTIJOIN INSCRIPCION

Variables y explicacion: No existe inscripcion asociada.

CRD9. Multiples relaciones

CRD:

$\{ \langle ne, ncu, cal \rangle \mid \text{ESTUDIANTE}(e, ne, co, ed, se, pr, ca, tu) \wedge \text{INSCRIPCION}(i, e, g, f, cal, es) \wedge \text{GRUPO}(g, cu, p, a, per, cupo, hor) \wedge \text{CURSO}(cu, ncu, cr, ti, ar, d) \}$

Equivalente en AR:

ESTUDIANTE JOIN INSCRIPCION JOIN GRUPO JOIN CURSO

Variables y explicacion: Une cuatro relaciones por sus claves.

CRD10. Cuantificadores anidados

CRD:

$$\{ \langle np \rangle \mid \text{PROFESOR}(p, np, co, gr, sa, dep, sup) \wedge \forall cid \forall cn \forall cr \forall ti \forall ar \forall d ((\text{CURSO}(cid, cn, cr, ti, ar, d) \wedge ar = \text{"Programacion"}) \rightarrow \exists gid \exists a \exists per \exists cupo \exists hor \text{GRUPO}(gid, cid, p, a, per, cupo, hor)) \}$$

Equivalente en AR:

Division profesor/cursos de Programacion

Variables y explicacion: Todo curso de Programacion tiene al menos un grupo impartido por ese profesor.

4.3 Ejercicio 3.3 - Equivalencia entre AR, CRT, CRD y SQL

Las siguientes 10 consultas complejas demuestran equivalencia entre los cuatro lenguajes. Cada una conserva la misma intencion semantica: AR expresa operadores, CRT expresa predicados sobre tuplas, CRD expresa predicados sobre dominios y SQL ofrece la ejecucion practica.

F1. Seleccion simple: estudiantes de ISC con promedio minimo de 8.5

Algebra relacional:

$$\pi_{\text{nombre}, \text{semestre}, \text{promedio}}(\sigma_{\text{id_carrera}=1 \wedge \text{promedio} \geq 8.5}(\text{ESTUDIANTE}))$$

Calculo relacional de tuplas (CRT):

$$\{ \langle e.\text{nombre}, e.\text{semestre}, e.\text{promedio} \rangle \mid \text{ESTUDIANTE}(e) \wedge e.\text{id_carrera}=1 \wedge e.\text{promedio} \geq 8.5 \}$$

Calculo relacional de dominios (CRD):

$$\{ \langle n, s, p \rangle \mid \exists id \exists co \exists ed \exists tu \text{ ESTUDIANTE}(id, n, co, ed, s, p, 1, tu) \wedge p \geq 8.5 \}$$

SQL equivalente:

```
SELECT nombre, semestre, promedio FROM estudiante WHERE id_carrera=1 AND promedio>=8.5 ORDER BY promedio DESC;
```

Explicacion: Los cuatro lenguajes aplican el mismo filtro sobre ESTUDIANTE y proyectan tres atributos.

Resultado ejemplo:

nombre	semestre	promedio
Julio Milan	5	9.1
Fernanda Lopez	6	8.8
Diego Martinez	5	8.6

F2. Proyeccion con seleccion: correos de estudiantes sin tutor

Algebra relacional:

$$\pi_{\text{nombre}, \text{correo}}(\sigma_{\text{id_tutor} \text{ IS NULL}}(\text{ESTUDIANTE}))$$

Calculo relacional de tuplas (CRT):

$$\{ \langle e.\text{nombre}, e.\text{correo} \rangle \mid \text{ESTUDIANTE}(e) \wedge e.\text{id_tutor} \text{ IS NULL} \}$$

Calculo relacional de dominios (CRD):

$$\{ \langle n, co \rangle \mid \exists id \exists ed \exists se \exists pr \exists ca \text{ ESTUDIANTE}(id, n, co, ed, se, pr, ca, \text{NULL}) \}$$

SQL equivalente:

```
SELECT nombre, correo FROM estudiante WHERE id_tutor IS NULL ORDER BY nombre;
```

Explicacion: La condicion NULL se expresa como atributo sin valor/tutor no existente.

Resultado ejemplo:

nombre	correo
Leonardo Reyes	leonardo.reyes@alumno.ipn.mx
Mariana Nunez	mariana.nunez@alumno.ipn.mx
Ximena Vargas	ximena.vargas@alumno.ipn.mx

F3. Union: personas academicas con correo institucional

Algebra relacional:

$$\pi_{\text{nombre}, \text{correo}}(\text{ESTUDIANTE}) \cup \pi_{\text{nombre}, \text{correo}}(\text{PROFESOR})$$

Calculo relacional de tuplas (CRT):

$$\{ \langle x, y \rangle \mid (\exists e(\text{ESTUDIANTE}(e) \wedge x=e.\text{nombre} \wedge y=e.\text{correo})) \vee (\exists p(\text{PROFESOR}(p) \wedge x=p.\text{nombre} \wedge y=p.\text{correo})) \}$$

Calculo relacional de dominios (CRD):

$$\{ \langle n, co \rangle \mid \exists id \exists ed \exists se \exists pr \exists ca \exists tu \text{ ESTUDIANTE}(id, n, co, ed, se, pr, ca, tu) \vee \exists pid \exists gr \exists sa \exists dep \exists sup \text{ PROFESOR}(pid, n, co, gr, sa, dep, sup) \}$$

SQL equivalente:

```
SELECT nombre, correo FROM estudiante UNION SELECT nombre, correo FROM profesor ORDER BY nombre;
```

Explicacion: La union requiere relaciones compatibles: nombre y correo.

Resultado ejemplo:

nombre	correo
Andrea Gomez	andrea.gomez@alumno.ipn.mx
Bruno Salazar	bruno.salazar@alumno.ipn.mx
Camila Sanchez	camila.sanchez@alumno.ipn.mx
Diego Martinez	diego.martinez@alumno.ipn.mx
Dra. Ana Torres	ana.torres@ipn.mx
Dra. Elena Navarro	elena.navarro@ipn.mx
Dra. Maria Castillo	maria.castillo@ipn.mx
Dra. Paola Herrera	paola.herrera@ipn.mx
Dra. Sofia Aguilar	sofia.aguilar@ipn.mx
Dra. Valeria Soto	valeria.soto@ipn.mx
... 22 filas mas	

F4. Interseccion: departamentos que tienen carreras y profesores

Algebra relacional:

$$\pi_{id_departamento}(\text{CARRERA}) \cap \pi_{id_departamento}(\text{PROFESOR})$$

Calculo relacional de tuplas (CRT):

$$\{ \langle d \rangle \mid \exists c(\text{CARRERA}(c) \wedge d=c.id_departamento) \wedge \exists p(\text{PROFESOR}(p) \wedge d=p.id_departamento) \}$$

Calculo relacional de dominios (CRD):

$$\{ \langle d \rangle \mid \exists cid \exists nc \exists niv \exists cred \text{ CARRERA}(cid, nc, niv, cred, d) \wedge \exists pid \exists np \exists co \exists gr \exists sa \exists sup \text{ PROFESOR}(pid, np, co, gr, sa, d, sup) \}$$

SQL equivalente:

```
SELECT id_departamento FROM carrera INTERSECT SELECT id_departamento FROM profesor ORDER BY id_departamento;
```

Explicacion: La interseccion conserva solo los id_departamento presentes en ambas proyecciones.

Resultado ejemplo:

id_departamento
1
3
4

F5. Diferencia: estudiantes sin ninguna inscripcion

Algebra relacional:

$$\pi_{id_estudiante, nombre}(\text{ESTUDIANTE}) - \pi_{id_estudiante, nombre}(\text{ESTUDIANTE JOIN INSCRIPCION})$$

Calculo relacional de tuplas (CRT):

$$\{ \langle e.id_estudiante, e.nombre \rangle \mid \text{ESTUDIANTE}(e) \wedge \neg \exists i(\text{INSCRIPCION}(i) \wedge i.id_estudiante=e.id_estudiante) \}$$

Calculo relacional de dominios (CRD):

$\{ \langle id, n \rangle \mid \exists co \exists ed \exists se \exists pr \exists ca \exists tu \text{ ESTUDIANTE}(id, n, co, ed, se, pr, ca, tu) \wedge \neg \exists ii \exists g \exists f \exists cal \exists es \text{ INSCRIPCION}(ii, id, g, f, cal, es) \}$

SQL equivalente:

```
SELECT e.id_estudiante, e.nombre FROM estudiante e WHERE NOT EXISTS (SELECT 1 FROM inscripcion i WHERE i.id_estudiante=e.id_estudiante) ORDER BY e.id_estudiante;
```

Explicacion: La diferencia se implementa con NOT EXISTS para conservar estudiantes sin coincidencias.

Resultado ejemplo:

id_estudiante	nombre
---------------	--------

F6. Join de dos tablas: estudiantes con carrera

Algebra relacional:

$\pi_{\text{ESTUDIANTE.nombre, CARRERA.nombre}}(\text{ESTUDIANTE JOIN CARRERA})$

Calculo relacional de tuplas (CRT):

$\{ \langle e.nombre, c.nombre \rangle \mid \text{ESTUDIANTE}(e) \wedge \text{CARRERA}(c) \wedge e.id_carrera=c.id_carrera \}$

Calculo relacional de dominios (CRD):

$\{ \langle ne, nc \rangle \mid \exists id \exists co \exists ed \exists se \exists pr \exists tu \exists niv \exists cred \exists dep \text{ ESTUDIANTE}(id, ne, co, ed, se, pr, ca, tu) \wedge \text{CARRERA}(ca, nc, niv, cred, dep) \}$

SQL equivalente:

```
SELECT e.nombre AS estudiante, c.nombre AS carrera FROM estudiante e JOIN carrera c ON e.id_carrera=c.id_carrera ORDER BY estudiante LIMIT 12;
```

Explicacion: Todas las formas unen por id_carrera y devuelven estudiante-carrera.

Resultado ejemplo:

estudiante	carrera
Andrea Gomez	Ingeniería en Inteligencia Artificial
Bruno Salazar	Ingeniería en Sistemas Computacionales
Camila Sanchez	Ingeniería de Software
Diego Martinez	Ingeniería en Sistemas Computacionales
Emiliano Ruiz	Ciencia de Datos
Fernanda Lopez	Ingeniería en Sistemas Computacionales
Ivan Campos	Ingeniería de Software
Julio Milan	Ingeniería en Sistemas Computacionales
Leonardo Reyes	Ciencia de Datos
Lucia Fuentes	Ciencia de Datos
... 2 filas mas	

F7. Join de tres o mas tablas: inscripciones con estudiante, grupo y curso

Algebra relacional:

$\pi_{e.nombre, c.nombre, i.calificacion}(\text{ESTUDIANTE JOIN INSCRIPCION JOIN GRUPO JOIN CURSO})$

Calculo relacional de tuplas (CRT):

$\{ \langle e.nombre, c.nombre, i.calificacion \rangle \mid \text{ESTUDIANTE}(e) \wedge \text{INSCRIPCION}(i) \wedge \text{GRUPO}(g) \wedge \text{CURSO}(c) \wedge i.id_estudiante=e.id_estudiante \wedge i.id_grupo=g.id_grupo \wedge g.id_curso=c.id_curso \}$

Calculo relacional de dominios (CRD):

{<ne,nc,cal> | ESTUDIANTE(e,ne,co,ed,se,pr,ca,tu) ^ INSCRIPCION(i,e,g,f,cal,es) ^ GRUPO(g,cu,p,a,per,cupo,hor) ^ CURSO(cu,nc,cr,ti,ar,d)}

SQL equivalente:

```
SELECT e.nombre AS estudiante, c.nombre AS curso, i.calificacion FROM estudiante e JOIN inscripcion i ON e.id_estudiante=i.id_estudiante JOIN grupo g ON i.id_grupo=g.id_grupo JOIN curso c ON g.id_curso=c.id_curso ORDER BY estudiante, curso LIMIT 12;
```

Explicacion: Es una cadena de reuniones por llaves foraneas.

Resultado ejemplo:

estudiante	curso	calificacion
Andrea Gomez	Algoritmos y Estructuras	8.6
Andrea Gomez	Aprendizaje Automatico	8.5
Andrea Gomez	Bases de Datos	8.2
Andrea Gomez	Calculo Vectorial	8.9
Andrea Gomez	Inteligencia Artificial	8.2
Andrea Gomez	Mineria de Datos	8.6
Bruno Salazar	Algoritmos y Estructuras	7.5
Bruno Salazar	Bases de Datos	7.2
Bruno Salazar	Calculo Vectorial	5.7
Camila Sanchez	Algebra Lineal	7.9
... 2 filas mas		

F8. Theta join: aulas cuya capacidad supera el cupo del grupo

Algebra relacional:

AULA JOIN _AULA.capacidad>GRUPO.cupo GRUPO

Calculo relacional de tuplas (CRT):

{<a.clave,g.id_grupo> | AULA(a) ^ GRUPO(g) ^ a.capacidad>g.cupo}

Calculo relacional de dominios (CRD):

{<cl,gid> | AULA(aid,cl,ed,cap,ti,proy) ^ GRUPO(gid,cu,p,au,per,cupo,hor) ^ cap>cupo}

SQL equivalente:

```
SELECT a.clave, a.capacidad, g.id_grupo, g.cupo FROM aula a JOIN grupo g ON a.capacidad > g.cupo ORDER BY a.clave, g.id_grupo LIMIT 12;
```

Explicacion: El predicado del join no es igualdad, sino una comparacion de capacidad.

Resultado ejemplo:

clave	capacidad	id_grupo	cupo
A101	35	5	30
A101	35	6	30
A101	35	7	25
A101	35	8	25
A101	35	13	30
A101	35	14	30
A101	35	15	28
A101	35	19	30
A101	35	20	25

clave	capacidad	id_grupo	cupo
A102	40	1	35
... 2 filas mas			

F9. Outer join: todos los profesores aunque no tengan grupo

Algebra relacional:

$\text{PROFESOR LEFT OUTER JOIN GRUPO}$

Calculo relacional de tuplas (CRT):

$\{ \langle p.\text{nombre}, g.\text{id_grupo} \rangle \mid \text{PROFESOR}(p) \wedge (\text{GRUPO}(g) \wedge g.\text{id_profesor} = p.\text{id_profesor} \vee \neg \exists g2 (\text{GRUPO}(g2) \wedge g2.\text{id_profesor} = p.\text{id_profesor})) \}$

Calculo relacional de dominios (CRD):

$\{ \langle np, gid \rangle \mid \text{PROFESOR}(pid, np, co, gr, sa, dep, sup) \wedge (\text{GRUPO}(gid, cu, pid, a, per, cupo, hor) \vee \neg \exists g \exists cu \exists a \exists per \exists cupo \exists hor \text{GRUPO}(g, cu, pid, a, per, cupo, hor)) \}$

SQL equivalente:

```
SELECT p.nombre AS profesor, g.id_grupo, g.periodo FROM profesor p LEFT JOIN grupo g ON p.id_profesor=g.id_profesor ORDER BY p.id_profesor, g.id_grupo LIMIT 12;
```

Explicacion: La reunion externa izquierda conserva todos los profesores del lado izquierdo.

Resultado ejemplo:

profesor	id_grupo	periodo
Dra. Ana Torres	1	2026-2
Dra. Ana Torres	2	2026-2
Dra. Ana Torres	12	2026-2
Dra. Ana Torres	18	2026-1
Mtro. Luis Ramirez	16	2026-1
Dra. Sofia Aguilar	7	2026-2
Dra. Sofia Aguilar	10	2026-2
Mtro. Carlos Vega	3	2026-2
Dra. Maria Castillo	5	2026-2
Dra. Maria Castillo	6	2026-2
... 2 filas mas		

F10. Anti-join: profesores sin grupo asignado

Algebra relacional:

$\text{PROFESOR ANTIJOIN GRUPO}$

Calculo relacional de tuplas (CRT):

$\{ \langle p.\text{id_profesor}, p.\text{nombre} \rangle \mid \text{PROFESOR}(p) \wedge \neg \exists g (\text{GRUPO}(g) \wedge g.\text{id_profesor} = p.\text{id_profesor}) \}$

Calculo relacional de dominios (CRD):

$\{ \langle pid, np \rangle \mid \exists co \exists gr \exists sa \exists dep \exists sup \text{PROFESOR}(pid, np, co, gr, sa, dep, sup) \wedge \neg \exists gid \exists cu \exists a \exists per \exists cupo \exists hor \text{GRUPO}(gid, cu, pid, a, per, cupo, hor) \}$

SQL equivalente:

```
SELECT p.id_profesor, p.nombre FROM profesor p WHERE NOT EXISTS (SELECT 1 FROM grupo g WHERE g.id_profesor=p.id_profesor) ORDER BY p.id_profesor;
```

Explicacion: El anti-join devuelve tuplas de PROFESOR sin coincidencia en GRUPO.

Resultado ejemplo:

id_profesor	nombre
-------------	--------

4.4 Ejercicio 3.4 - Seguridad y completitud relacional

4.4.1 Expresiones seguras

Una expresion del calculo relacional es segura cuando garantiza un resultado finito y solo devuelve valores provenientes del dominio activo de la base de datos. Las expresiones inseguras intentan construir conjuntos sobre todo el universo posible, no solo sobre datos almacenados.

Insegura	Problema	Correccion segura
$\{t \mid \neg \text{ESTUDIANTE}(t)\}$	Genera todo lo que no es estudiante; universo infinito.	$\{t \mid \text{ESTUDIANTE}(t) \wedge \neg \exists i(\text{INSCRIPCION}(i) \wedge i.\text{id_estudiante}=t.\text{id_estudiante})\}$
$\{x \mid x > 0\}$	No limita x a una relacion de la base.	$\{x \mid \exists e(\text{ESTUDIANTE}(e) \wedge x=e.\text{edad} \wedge x>0)\}$
$\{t \mid \neg \text{CURSO}(t)\}$	Incluye cualquier tupla que no sea curso.	$\{t \mid \text{CURSO}(t) \wedge t.\text{tipo}=\text{"Optativo"}\}$
$\{x \mid x \neq \text{"Computacion"}\}$	Puede producir infinitos valores distintos.	$\{x \mid \exists d(\text{DEPARTAMENTO}(d) \wedge x=d.\text{nombre} \wedge x \neq \text{"Computacion"})\}$
$\{(x,y) \mid x=y\}$	No hay dominio activo ni relacion de origen.	$\{(e.\text{nombre}, e.\text{correo}) \mid \text{ESTUDIANTE}(e) \wedge e.\text{nombre} \text{ IS NOT NULL} \wedge e.\text{correo} \text{ IS NOT NULL}\}$

4.4.2 Completitud relacional

El algebra relacional, el calculo relacional de tuplas seguro y el calculo relacional de dominios seguro son equivalentes en poder expresivo para consultas relacionales finitas de primer orden. Esto significa que una consulta formulada correctamente en uno de estos lenguajes puede traducirse a los otros sin cambiar su resultado.

Tipo	AR	CRT seguro	CRD seguro
Seleccion	$\sigma_{\text{promedio} \geq 9}(\text{ESTUDIANTE})$	$\{t \mid \text{ESTUDIANTE}(t) \wedge t.\text{promedio} \geq 9\}$	$\{(id,n,co,ed,se,pr,ca,tu) \mid \text{ESTUDIANTE}(id,n,co,ed,se,pr,ca,tu) \wedge pr \geq 9\}$
Join	$\text{ESTUDIANTE} \bowtie \text{CARRERA}$	$\{(e,c) \mid \text{ESTUDIANTE}(e) \wedge \text{CARRERA}(c) \wedge e.\text{id_carrera}=c.\text{id_carrera}\}$	$\{(ne,nc) \mid \text{ESTUDIANTE}(id,ne,co,ed,se,pr,ca,tu) \wedge \text{CARRERA}(ca,nc,niv,cred,dep)\}$
Division/universal	$R \div S$	$\{e \mid \text{ESTUDIANTE}(e) \wedge \forall c(\text{CURSO_OBLIGATORIO}(c) \rightarrow \text{Inscrito}(e,c))\}$	$\{(id,n) \mid \text{ESTUDIANTE}(id,n,...) \wedge \forall cid(\text{CURSO_OBLIGATORIO}(cid) \rightarrow \text{INSCRITO}(id,cid))\}$

Limitaciones: estos lenguajes de primer orden no expresan de forma natural consultas recursivas ilimitadas como caminos de longitud arbitraria en grafos. Tampoco modelan agregaciones complejas dentro del calculo relacional puro; para eso se usa algebra relacional extendida o SQL con GROUP BY, funciones agregadas y, si se requiere, consultas recursivas.

5. Ejercicio final - Caso integrador

El caso integrador se resuelve con el mismo esquema de ocho tablas y mas de 100 tuplas. A continuacion se presentan 20 consultas complejas, distribuidas como solicita la practica: 5 con operadores basicos, 5 con reuniones, 5 con agrupacion/agregacion, 3 con division y 2 con cuantificadores universales.

Bloque	Consultas
Operadores basicos	F1-F5
Reuniones	F6-F10
Agrupacion y agregacion	F11-F15
Division	F16-F18
Cuantificadores universales	F19-F20

F1. Seleccion simple: estudiantes de ISC con promedio minimo de 8.5

Algebra relacional:

$\pi_{\text{nombre}, \text{semestre}, \text{promedio}}(\sigma_{\text{id_carrera}=1 \wedge \text{promedio} \geq 8.5}(\text{ESTUDIANTE}))$

Calculo relacional de tuplas (CRT):

$\{ \langle e.\text{nombre}, e.\text{semestre}, e.\text{promedio} \rangle \mid \text{ESTUDIANTE}(e) \wedge e.\text{id_carrera}=1 \wedge e.\text{promedio} \geq 8.5 \}$

Calculo relacional de dominios (CRD):

$\{ \langle n, s, p \rangle \mid \exists id \exists co \exists ed \exists tu \text{ ESTUDIANTE}(id, n, co, ed, s, p, 1, tu) \wedge p \geq 8.5 \}$

SQL equivalente:

`SELECT nombre, semestre, promedio FROM estudiante WHERE id_carrera=1 AND promedio>=8.5 ORDER BY promedio DESC;`

Explicacion: Los cuatro lenguajes aplican el mismo filtro sobre ESTUDIANTE y proyectan tres atributos.

Resultado ejemplo:

nombre	semestre	promedio
Julio Milan	5	9.1
Fernanda Lopez	6	8.8
Diego Martinez	5	8.6

F2. Proyeccion con seleccion: correos de estudiantes sin tutor

Algebra relacional:

$\pi_{\text{nombre}, \text{correo}}(\sigma_{\text{id_tutor} \text{ IS NULL}}(\text{ESTUDIANTE}))$

Calculo relacional de tuplas (CRT):

$\{ \langle e.\text{nombre}, e.\text{correo} \rangle \mid \text{ESTUDIANTE}(e) \wedge e.\text{id_tutor} \text{ IS NULL} \}$

Calculo relacional de dominios (CRD):

$\{ \langle n, co \rangle \mid \exists id \exists ed \exists se \exists pr \exists ca \text{ ESTUDIANTE}(id, n, co, ed, se, pr, ca, \text{NULL}) \}$

SQL equivalente:

`SELECT nombre, correo FROM estudiante WHERE id_tutor IS NULL ORDER BY nombre;`

Explicacion: La condicion NULL se expresa como atributo sin valor/tutor no existente.

Resultado ejemplo:

nombre	correo
Leonardo Reyes	leonardo.reyes@alumno.ipn.mx
Mariana Nunez	mariana.nunez@alumno.ipn.mx

nombre	correo
Ximena Vargas	ximena.vargas@alumno.ipn.mx

F3. Union: personas academicas con correo institucional

Algebra relacional:

$$\pi_{\text{nombre, correo}}(\text{ESTUDIANTE}) \cup \pi_{\text{nombre, correo}}(\text{PROFESOR})$$

Calculo relacional de tuplas (CRT):

$$\{ \langle x, y \rangle \mid (\exists e(\text{ESTUDIANTE}(e) \wedge x=e.\text{nombre} \wedge y=e.\text{correo})) \vee (\exists p(\text{PROFESOR}(p) \wedge x=p.\text{nombre} \wedge y=p.\text{correo})) \}$$

Calculo relacional de dominios (CRD):

$$\{ \langle n, co \rangle \mid \exists id \exists ed \exists se \exists pr \exists ca \exists tu \text{ ESTUDIANTE}(id, n, co, ed, se, pr, ca, tu) \vee \exists pid \exists gr \exists sa \exists dep \exists sup \text{ PROFESOR}(pid, n, co, gr, sa, dep, sup) \}$$

SQL equivalente:

```
SELECT nombre, correo FROM estudiante UNION SELECT nombre, correo FROM profesor ORDER BY nombre;
```

Explicacion: La union requiere relaciones compatibles: nombre y correo.

Resultado ejemplo:

nombre	correo
Andrea Gomez	andrea.gomez@alumno.ipn.mx
Bruno Salazar	bruno.salazar@alumno.ipn.mx
Camila Sanchez	camila.sanchez@alumno.ipn.mx
Diego Martinez	diego.martinez@alumno.ipn.mx
Dra. Ana Torres	ana.torres@ipn.mx
Dra. Elena Navarro	elena.navarro@ipn.mx
Dra. Maria Castillo	maria.castillo@ipn.mx
Dra. Paola Herrera	paola.herrera@ipn.mx
Dra. Sofia Aguilar	sofia.aguilar@ipn.mx
Dra. Valeria Soto	valeria.soto@ipn.mx
... 22 filas mas	

F4. Interseccion: departamentos que tienen carreras y profesores

Algebra relacional:

$$\pi_{\text{id_departamento}}(\text{CARRERA}) \cap \pi_{\text{id_departamento}}(\text{PROFESOR})$$

Calculo relacional de tuplas (CRT):

$$\{ \langle d \rangle \mid \exists c(\text{CARRERA}(c) \wedge d=c.\text{id_departamento}) \wedge \exists p(\text{PROFESOR}(p) \wedge d=p.\text{id_departamento}) \}$$

Calculo relacional de dominios (CRD):

$$\{ \langle d \rangle \mid \exists cid \exists nc \exists niv \exists cred \text{ CARRERA}(cid, nc, niv, cred, d) \wedge \exists pid \exists np \exists co \exists gr \exists sa \exists sup \text{ PROFESOR}(pid, np, co, gr, sa, d, sup) \}$$

SQL equivalente:

```
SELECT id_departamento FROM carrera INTERSECT SELECT id_departamento FROM profesor ORDER BY id_departamento;
```

Explicacion: La interseccion conserva solo los id_departamento presentes en ambas proyecciones.

Resultado ejemplo:

id_departamento
1

id_departamento
3
4

F5. Diferencia: estudiantes sin ninguna inscripcion

Algebra relacional:

$$\pi_{id_estudiante, nombre}(ESTUDIANTE) - \pi_{id_estudiante, nombre}(ESTUDIANTE \text{ JOIN } INSCRIPCION)$$

Calculo relacional de tuplas (CRT):

$$\{ \langle e.id_estudiante, e.nombre \rangle \mid ESTUDIANTE(e) \wedge \neg \exists i (INSCRIPCION(i) \wedge i.id_estudiante = e.id_estudiante) \}$$

Calculo relacional de dominios (CRD):

$$\{ \langle id, n \rangle \mid \exists co \exists ed \exists se \exists pr \exists ca \exists tu \text{ ESTUDIANTE}(id, n, co, ed, se, pr, ca, tu) \wedge \neg \exists ii \exists g \exists f \exists cal \exists es \text{ INSCRIPCION}(ii, id, g, f, cal, es) \}$$

SQL equivalente:

```
SELECT e.id_estudiante, e.nombre FROM estudiante e WHERE NOT EXISTS (SELECT 1 FROM inscripcion i WHERE i.id_estudiante=e.id_estudiante) ORDER BY e.id_estudiante;
```

Explicacion: La diferencia se implementa con NOT EXISTS para conservar estudiantes sin coincidencias.

Resultado ejemplo:

id_estudiante	nombre
---------------	--------

F6. Join de dos tablas: estudiantes con carrera

Algebra relacional:

$$\pi_{ESTUDIANTE.nombre, CARRERA.nombre}(ESTUDIANTE \text{ JOIN } CARRERA)$$

Calculo relacional de tuplas (CRT):

$$\{ \langle e.nombre, c.nombre \rangle \mid ESTUDIANTE(e) \wedge CARRERA(c) \wedge e.id_carrera = c.id_carrera \}$$

Calculo relacional de dominios (CRD):

$$\{ \langle ne, nc \rangle \mid \exists id \exists co \exists ed \exists se \exists pr \exists tu \exists niv \exists cred \exists dep \text{ ESTUDIANTE}(id, ne, co, ed, se, pr, ca, tu) \wedge CARRERA(ca, nc, niv, cred, dep) \}$$

SQL equivalente:

```
SELECT e.nombre AS estudiante, c.nombre AS carrera FROM estudiante e JOIN carrera c ON e.id_carrera=c.id_carrera ORDER BY estudiante LIMIT 12;
```

Explicacion: Todas las formas unen por id_carrera y devuelven estudiante-carrera.

Resultado ejemplo:

estudiante	carrera
Andrea Gomez	Ingenieria en Inteligencia Artificial
Bruno Salazar	Ingenieria en Sistemas Computacionales
Camila Sanchez	Ingenieria de Software
Diego Martinez	Ingenieria en Sistemas Computacionales
Emiliano Ruiz	Ciencia de Datos
Fernanda Lopez	Ingenieria en Sistemas Computacionales
Ivan Campos	Ingenieria de Software
Julio Milan	Ingenieria en Sistemas Computacionales
Leonardo Reyes	Ciencia de Datos
Lucia Fuentes	Ciencia de Datos

estudiante	carrera
... 2 filas mas	

F7. Join de tres o mas tablas: inscripciones con estudiante, grupo y curso

Algebra relacional:

$$\pi_{e.nombre, c.nombre, i.calificacion}(ESTUDIANTE \text{ JOIN } INSCRIPCION \text{ JOIN } GRUPO \text{ JOIN } CURSO)$$

Calculo relacional de tuplas (CRT):

$$\{ \langle e.nombre, c.nombre, i.calificacion \rangle \mid ESTUDIANTE(e) \wedge INSCRIPCION(i) \wedge GRUPO(g) \wedge CURSO(c) \wedge i.id_estudiante=e.id_estudiante \wedge i.id_grupo=g.id_grupo \wedge g.id_curso=c.id_curso \}$$

Calculo relacional de dominios (CRD):

$$\{ \langle ne, nc, cal \rangle \mid ESTUDIANTE(e, ne, co, ed, se, pr, ca, tu) \wedge INSCRIPCION(i, e, g, f, cal, es) \wedge GRUPO(g, cu, p, a, per, cupo, hor) \wedge CURSO(cu, nc, cr, ti, ar, d) \}$$

SQL equivalente:

```
SELECT e.nombre AS estudiante, c.nombre AS curso, i.calificacion FROM estudiante e JOIN inscripcion i ON e.id_estudiante=i.id_estudiante JOIN grupo g ON i.id_grupo=g.id_grupo JOIN curso c ON g.id_curso=c.id_curso ORDER BY estudiante, curso LIMIT 12;
```

Explicacion: Es una cadena de reuniones por llaves foraneas.

Resultado ejemplo:

estudiante	curso	calificacion
Andrea Gomez	Algoritmos y Estructuras	8.6
Andrea Gomez	Aprendizaje Automatico	8.5
Andrea Gomez	Bases de Datos	8.2
Andrea Gomez	Calculo Vectorial	8.9
Andrea Gomez	Inteligencia Artificial	8.2
Andrea Gomez	Mineria de Datos	8.6
Bruno Salazar	Algoritmos y Estructuras	7.5
Bruno Salazar	Bases de Datos	7.2
Bruno Salazar	Calculo Vectorial	5.7
Camila Sanchez	Algebra Lineal	7.9
... 2 filas mas		

F8. Theta join: aulas cuya capacidad supera el cupo del grupo

Algebra relacional:

$$AULA \text{ JOIN } _AULA.capacidad > GRUPO.cupo \text{ GRUPO}$$

Calculo relacional de tuplas (CRT):

$$\{ \langle a.clave, g.id_grupo \rangle \mid AULA(a) \wedge GRUPO(g) \wedge a.capacidad > g.cupo \}$$

Calculo relacional de dominios (CRD):

$$\{ \langle cl, gid \rangle \mid AULA(aid, cl, ed, cap, ti, proy) \wedge GRUPO(gid, cu, p, au, per, cupo, hor) \wedge cap > cupo \}$$

SQL equivalente:

```
SELECT a.clave, a.capacidad, g.id_grupo, g.cupo FROM aula a JOIN grupo g ON a.capacidad > g.cupo ORDER BY a.clave, g.id_grupo LIMIT 12;
```

Explicacion: El predicado del join no es igualdad, sino una comparacion de capacidad.

Resultado ejemplo:

clave	capacidad	id_grupo	cupo
A101	35	5	30
A101	35	6	30
A101	35	7	25
A101	35	8	25
A101	35	13	30
A101	35	14	30
A101	35	15	28
A101	35	19	30
A101	35	20	25
A102	40	1	35
... 2 filas mas			

F9. Outer join: todos los profesores aunque no tengan grupo

Algebra relacional:

PROFESOR LEFT OUTER JOIN GRUPO

Calculo relacional de tuplas (CRT):

$\{ \langle p.nombre, g.id_grupo \rangle \mid \text{PROFESOR}(p) \wedge (\text{GRUPO}(g) \wedge g.id_profesor=p.id_profesor \vee \neg \exists g2(\text{GRUPO}(g2) \wedge g2.id_profesor=p.id_profesor)) \}$

Calculo relacional de dominios (CRD):

$\{ \langle np, gid \rangle \mid \text{PROFESOR}(pid, np, co, gr, sa, dep, sup) \wedge (\text{GRUPO}(gid, cu, pid, a, per, cupo, hor) \vee \neg \exists g \exists cu \exists a \exists per \exists cupo \exists hor \text{ GRUPO}(g, cu, pid, a, per, cupo, hor)) \}$

SQL equivalente:

```
SELECT p.nombre AS profesor, g.id_grupo, g.periodo FROM profesor p LEFT JOIN grupo g ON p.id_profesor=g.id_profesor ORDER BY p.id_profesor, g.id_grupo
LIMIT 12;
```

Explicacion: La reunion externa izquierda conserva todos los profesores del lado izquierdo.

Resultado ejemplo:

profesor	id_grupo	periodo
Dra. Ana Torres	1	2026-2
Dra. Ana Torres	2	2026-2
Dra. Ana Torres	12	2026-2
Dra. Ana Torres	18	2026-1
Mtro. Luis Ramirez	16	2026-1
Dra. Sofia Aguilar	7	2026-2
Dra. Sofia Aguilar	10	2026-2
Mtro. Carlos Vega	3	2026-2
Dra. Maria Castillo	5	2026-2
Dra. Maria Castillo	6	2026-2
... 2 filas mas		

F10. Anti-join: profesores sin grupo asignado

Algebra relacional:

PROFESOR ANTIJOIN GRUPO

Calculo relacional de tuplas (CRT):

$\{ \langle p.id_profesor, p.nombre \rangle \mid PROFESOR(p) \wedge \neg \exists g(GRUPO(g) \wedge g.id_profesor=p.id_profesor) \}$

Calculo relacional de dominios (CRD):

$\{ \langle pid, np \rangle \mid \exists co \exists gr \exists sa \exists dep \exists sup PROFESOR(pid, np, co, gr, sa, dep, sup) \wedge \neg \exists gid \exists cu \exists a \exists per \exists cupo \exists hor GRUPO(gid, cu, pid, a, per, cupo, hor) \}$

SQL equivalente:

```
SELECT p.id_profesor, p.nombre FROM profesor p WHERE NOT EXISTS (SELECT 1 FROM grupo g WHERE g.id_profesor=p.id_profesor) ORDER BY p.id_profesor;
```

Explicacion: El anti-join devuelve tuplas de PROFESOR sin coincidencia en GRUPO.

Resultado ejemplo:

id_profesor	nombre
-------------	--------

F11. Agregacion: total de estudiantes por carrera

Algebra relacional:

$G_CARRERA.nombre \ F_COUNT(ESTUDIANTE.id_estudiante) \rightarrow total(ESTUDIANTE \ JOIN \ CARRERA)$

Calculo relacional de tuplas (CRT):

$\{ \langle c.nombre, n \rangle \mid CARRERA(c) \wedge n=COUNT(\{e \mid ESTUDIANTE(e) \wedge e.id_carrera=c.id_carrera\}) \}$

Calculo relacional de dominios (CRD):

$\{ \langle nc, n \rangle \mid CARRERA(ca, nc, niv, cred, dep) \wedge n=COUNT(\{id \mid ESTUDIANTE(id, ne, co, ed, se, pr, ca, tu)\}) \}$

SQL equivalente:

```
SELECT c.nombre AS carrera, COUNT(e.id_estudiante) AS total FROM carrera c LEFT JOIN estudiante e ON c.id_carrera=e.id_carrera GROUP BY c.nombre ORDER BY carrera;
```

Explicacion: El resultado agrupa estudiantes por carrera y cuenta miembros.

Resultado ejemplo:

carrera	total
Ciencia de Datos	4
Ingenieria de Software	4
Ingenieria en Inteligencia Artificial	4
Ingenieria en Sistemas Computacionales	5
Seguridad Informatica	3

F12. Agregacion: promedio de calificaciones por curso

Algebra relacional:

$G_CURSO.nombre \ F_AVG(calificacion) \rightarrow promedio(CURSO \ JOIN \ GRUPO \ JOIN \ INSCRIPCION)$

Calculo relacional de tuplas (CRT):

$\{ \langle c.nombre, p \rangle \mid CURSO(c) \wedge p=AVG(\{i.calificacion \mid INSCRIPCION(i) \wedge GRUPO(g) \wedge i.id_grupo=g.id_grupo \wedge g.id_curso=c.id_curso\}) \}$

Calculo relacional de dominios (CRD):

$\{ \langle nc, prom \rangle \mid CURSO(cu, nc, cr, ti, ar, d) \wedge prom=AVG(\{cal \mid INSCRIPCION(i, e, g, f, cal, es) \wedge GRUPO(g, cu, pr, a, per, cupo, hor)\}) \}$

SQL equivalente:

```
SELECT c.nombre AS curso, ROUND(AVG(i.calificacion),2) AS promedio FROM curso c JOIN grupo g ON c.id_curso=g.id_curso JOIN inscripcion i ON i.id_grupo=g.id_grupo GROUP BY c.nombre ORDER BY promedio DESC;
```

Explicacion: Une curso-grupo-inscripcion y calcula AVG por curso.

Resultado ejemplo:

curso	promedio
Mineria de Datos	8.6
Inteligencia Artificial	8.33
Redes de Computadoras	8.24
Aprendizaje Automatico	8.18
Sistemas Operativos	8.15
Programacion Web	8.1
Bases de Datos	7.99
Algebra Lineal	7.97
Algoritmos y Estructuras	7.86
Calculo Vectorial	7.58
... 5 filas mas	

F13. Agrupacion con HAVING: grupos con mas de cinco inscritos

Algebra relacional:

$$\sigma_{total > 5}(G_id_grupo \ F_COUNT(*) \rightarrow total(INSCRIPCION))$$

Calculo relacional de tuplas (CRT):

$$\{ \langle g, n \rangle \mid GRUPO(g) \wedge n = COUNT(\{i \mid INSCRIPCION(i) \wedge i.id_grupo = g.id_grupo\}) \wedge n > 5 \}$$

Calculo relacional de dominios (CRD):

$$\{ \langle gid, n \rangle \mid GRUPO(gid, cu, p, a, per, cupo, hor) \wedge n = COUNT(\{i \mid INSCRIPCION(i, e, gid, f, cal, es)\}) \wedge n > 5 \}$$

SQL equivalente:

```
SELECT id_grupo, COUNT(*) AS total FROM inscripcion GROUP BY id_grupo HAVING COUNT(*) > 5 ORDER BY total DESC, id_grupo;
```

Explicacion: HAVING filtra despues de agrupar.

Resultado ejemplo:

id_grupo	total
1	14
2	13
3	10
10	9
4	8
5	6
6	6
7	6
12	6

F14. Multiples agregaciones: salario promedio, maximo y minimo por departamento

Algebra relacional:

$$G_DEPARTAMENTO.nombre \ F_AVG(salario), MAX(salario), MIN(salario)(PROFESOR \ JOIN \ DEPARTAMENTO)$$

Calculo relacional de tuplas (CRT):

$\{ \langle d.nombre, avg, max, min \rangle \mid DEPARTAMENTO(d) \wedge avg=AVG(\{ p.salario \mid PROFESOR(p) \wedge p.id_departamento=d.id_departamento \}) \wedge max=MAX(\dots) \wedge min=MIN(\dots) \}$

Calculo relacional de dominios (CRD):

$\{ \langle nd, avg, max, min \rangle \mid DEPARTAMENTO(dep, nd, ed, tel) \wedge avg=AVG(\{ sa \mid PROFESOR(pid, np, co, gr, sa, dep, sup) \}) \wedge max=MAX(\{ sa \mid PROFESOR(pid, np, co, gr, sa, dep, sup) \}) \wedge min=MIN(\{ sa \mid PROFESOR(pid, np, co, gr, sa, dep, sup) \}) \}$

SQL equivalente:

```
SELECT d.nombre AS departamento, ROUND(AVG(p.salario),2) AS salario_promedio, MAX(p.salario) AS salario_maximo, MIN(p.salario) AS
salario_minimo FROM
departamento d JOIN profesor p ON d.id_departamento=p.id_departamento GROUP BY d.nombre ORDER BY d.nombre;
```

Explicacion: Agrupa profesores por departamento y calcula tres funciones.

Resultado ejemplo:

departamento	salario_promedio	salario_maximo	salario_minimo
Computacion	35375.0	42000.0	28000.0
Gestion Academica	33500.0	33500.0	33500.0
Inteligencia Artificial	45000.0	46000.0	44000.0
Matematicas	37500.0	41000.0	34000.0
Sistemas	36000.0	43000.0	29000.0

F15. Agregacion sobre reunion: aprobados por periodo

Algebra relacional:

$G_periodo \ F_COUNT(*) \rightarrow aprobados(\sigma_{estado='Aprobado'}(INSCRIPCION \ JOIN \ GRUPO))$

Calculo relacional de tuplas (CRT):

$\{ \langle g.periodo, n \rangle \mid GRUPO(g) \wedge n=COUNT(\{ i \mid INSCRIPCION(i) \wedge i.id_grupo=g.id_grupo \wedge i.estado='Aprobado' \}) \}$

Calculo relacional de dominios (CRD):

$\{ \langle per, n \rangle \mid GRUPO(gid, cu, p, a, per, cupo, hor) \wedge n=COUNT(\{ i \mid INSCRIPCION(i, e, gid, f, cal, 'Aprobado' \}) \}$

SQL equivalente:

```
SELECT g.periodo, COUNT(*) AS aprobados FROM grupo g JOIN inscripcion i ON g.id_grupo=i.id_grupo WHERE i.estado='Aprobado' GROUP BY g.periodo
ORDER BY
g.periodo;
```

Explicacion: Primero se une inscripcion con grupo, se filtran aprobados y luego se agrupa por periodo.

Resultado ejemplo:

periodo	aprobados
2026-1	6
2026-2	94

F16. Division: estudiantes inscritos en todos los cursos obligatorios

Algebra relacional:

$R(id_estudiante, id_curso) \ DIV \ S(id_curso), \ R=\pi_{id_estudiante, id_curso}(INSCRIPCION \ JOIN \ GRUPO), \ S=\pi_{id_curso}(\sigma_{tipo='Obligatorio'}(CURSO))$

Calculo relacional de tuplas (CRT):

$\{ \langle e.id_estudiante, e.nombre \rangle \mid ESTUDIANTE(e) \wedge \forall c((CURSO(c) \wedge c.tipo='Obligatorio') \rightarrow \exists i \exists g (INSCRIPCION(i) \wedge GRUPO(g) \wedge i.id_estudiante=e.id_estudiante \wedge i.id_grupo=g.id_grupo \wedge g.id_curso=c.id_curso)) \}$

Calculo relacional de dominios (CRD):

$$\{ \langle id, n \rangle \mid ESTUDIANTE(id, n, co, ed, se, pr, ca, tu) \wedge \forall cid \forall cn \forall cr \forall ar \forall dep ((CURSO(cid, cn, cr, 'Obligatorio', ar, dep)) \rightarrow \exists ii \exists gid \exists f \exists cal \exists es \exists p \exists a \exists per \exists cupo \exists hor (INSCRIPCION(ii, id, gid, f, cal, es) \wedge GRUPO(gid, cid, p, a, per, cupo, hor))) \}$$

SQL equivalente:

```
SELECT e.id_estudiante, e.nombre FROM estudiante e WHERE NOT EXISTS (SELECT 1 FROM curso c WHERE c.tipo='Obligatorio' AND NOT EXISTS (SELECT 1 FROM inscripcion i JOIN grupo g ON i.id_grupo=g.id_grupo WHERE i.id_estudiante=e.id_estudiante AND g.id_curso=c.id_curso)) ORDER BY e.id_estudiante;
```

Explicacion: NOT EXISTS doble implementa el cuantificador universal de la division.

Resultado ejemplo:

id_estudiante	nombre
---------------	--------

F17. Division: estudiantes inscritos en todos los cursos del area Base de Datos

Algebra relacional:

$$R(id_estudiante, id_curso) \text{ DIV } \pi_{id_curso}(\sigma_{area='Base de Datos'}(CURSO))$$

Calculo relacional de tuplas (CRT):

$$\{ \langle e.id_estudiante, e.nombre \rangle \mid ESTUDIANTE(e) \wedge \forall c ((CURSO(c) \wedge c.area='Base de Datos') \rightarrow \exists i \exists g (INSCRIPCION(i) \wedge GRUPO(g) \wedge i.id_estudiante=e.id_estudiante \wedge i.id_grupo=g.id_grupo \wedge g.id_curso=c.id_curso)) \}$$

Calculo relacional de dominios (CRD):

$$\{ \langle id, n \rangle \mid ESTUDIANTE(id, n, co, ed, se, pr, ca, tu) \wedge \forall cid \forall cn \forall cr \forall ti \forall dep ((CURSO(cid, cn, cr, ti, 'Base de Datos', dep)) \rightarrow \exists ii \exists gid \exists f \exists cal \exists es \exists p \exists a \exists per \exists cupo \exists hor (INSCRIPCION(ii, id, gid, f, cal, es) \wedge GRUPO(gid, cid, p, a, per, cupo, hor))) \}$$

SQL equivalente:

```
SELECT e.id_estudiante, e.nombre FROM estudiante e WHERE NOT EXISTS (SELECT 1 FROM curso c WHERE c.area='Base de Datos' AND NOT EXISTS (SELECT 1 FROM inscripcion i JOIN grupo g ON i.id_grupo=g.id_grupo WHERE i.id_estudiante=e.id_estudiante AND g.id_curso=c.id_curso)) ORDER BY e.id_estudiante;
```

Explicacion: Misma forma de division, pero el divisor son cursos de Base de Datos.

Resultado ejemplo:

id_estudiante	nombre
1	Julio Milan
2	Fernanda Lopez
3	Diego Martinez
4	Andrea Gomez
6	Natalia Perez
12	Regina Ortega
16	Renata Rojas
18	Lucia Fuentes

F18. Division: profesores que imparten todos los cursos de Programacion

Algebra relacional:

$$\pi_{id_profesor, nombre}(PROFESOR) \text{ JOIN } ((\pi_{id_profesor, id_curso}(GRUPO)) \text{ DIV } \pi_{id_curso}(\sigma_{area='Programacion'}(CURSO)))$$

Calculo relacional de tuplas (CRT):

$$\{ \langle p.id_profesor, p.nombre \rangle \mid PROFESOR(p) \wedge \forall c ((CURSO(c) \wedge c.area='Programacion') \rightarrow \exists g (GRUPO(g) \wedge g.id_profesor=p.id_profesor \wedge g.id_curso=c.id_curso)) \}$$

Calculo relacional de dominios (CRD):

$$\{ \langle \text{pid}, \text{np} \rangle \mid \text{PROFESOR}(\text{pid}, \text{np}, \text{co}, \text{gr}, \text{sa}, \text{dep}, \text{sup}) \wedge \forall \text{cid} \forall \text{cn} \forall \text{cr} \forall \text{ti} \forall \text{d} ((\text{CURSO}(\text{cid}, \text{cn}, \text{cr}, \text{ti}, \text{'Programacion'}, \text{d})) \rightarrow \exists \text{gid} \exists \text{a} \exists \text{per} \exists \text{cupo} \exists \text{hor} \text{GRUPO}(\text{gid}, \text{cid}, \text{pid}, \text{a}, \text{per}, \text{cupo}, \text{hor})) \}$$

SQL equivalente:

```
SELECT p.id_profesor, p.nombre FROM profesor p WHERE NOT EXISTS (SELECT 1 FROM curso c WHERE c.area='Programacion' AND NOT EXISTS (SELECT 1 FROM grupo g WHERE g.id_profesor=p.id_profesor AND g.id_curso=c.id_curso)) ORDER BY p.id_profesor;
```

Explicacion: El profesor debe cubrir cada curso del conjunto divisor Programacion.

Resultado ejemplo:

id_profesor	nombre
1	Dra. Ana Torres

F19. Cuantificador universal: estudiantes que aprobaron todos sus cursos inscritos

Algebra relacional:

$$\pi_{\text{nombre}}(\text{ESTUDIANTE}) - \pi_{\text{nombre}}(\text{ESTUDIANTE JOIN } \sigma_{\text{calificacion} < 6}(\text{INSCRIPCION}))$$

Calculo relacional de tuplas (CRT):

$$\{ \langle \text{e.nombre} \rangle \mid \text{ESTUDIANTE}(\text{e}) \wedge \forall \text{i} ((\text{INSCRIPCION}(\text{i}) \wedge \text{i.id_estudiante} = \text{e.id_estudiante}) \rightarrow \text{i.calificacion} \geq 6) \}$$

Calculo relacional de dominios (CRD):

$$\{ \langle \text{n} \rangle \mid \text{ESTUDIANTE}(\text{id}, \text{n}, \text{co}, \text{ed}, \text{se}, \text{pr}, \text{ca}, \text{tu}) \wedge \forall \text{ii} \forall \text{g} \forall \text{f} \forall \text{cal} \forall \text{es} ((\text{INSCRIPCION}(\text{ii}, \text{id}, \text{g}, \text{f}, \text{cal}, \text{es})) \rightarrow \text{cal} \geq 6) \}$$

SQL equivalente:

```
SELECT e.nombre FROM estudiante e WHERE NOT EXISTS (SELECT 1 FROM inscripcion i WHERE i.id_estudiante=e.id_estudiante AND i.calificacion < 6) ORDER BY e.nombre;
```

Explicacion: El universal se expresa como ausencia de contraejemplos reprobados.

Resultado ejemplo:

nombre
Andrea Gomez
Camila Sanchez
Diego Martinez
Fernanda Lopez
Ivan Campos
Julio Milan
Leonardo Reyes
Lucia Fuentes
Mariana Nunez
Mateo Hernandez
... 8 filas mas

F20. Cuantificador universal: carreras donde todos sus estudiantes tienen promedio minimo de 8

Algebra relacional:

$$\pi_{\text{nombre}}(\text{CARRERA}) - \pi_{\text{nombre}}(\text{CARRERA JOIN } \sigma_{\text{promedio} < 8}(\text{ESTUDIANTE}))$$

Calculo relacional de tuplas (CRT):

$$\{ \langle \text{c.nombre} \rangle \mid \text{CARRERA}(\text{c}) \wedge \forall \text{e} ((\text{ESTUDIANTE}(\text{e}) \wedge \text{e.id_carrera} = \text{c.id_carrera}) \rightarrow \text{e.promedio} \geq 8) \}$$

Calculo relacional de dominios (CRD):

$$\{ \langle nc \rangle \mid CARRERA(ca, nc, niv, cred, dep) \wedge \forall id \forall ne \forall co \forall ed \forall se \forall pr \forall tu ((ESTUDIANTE(id, ne, co, ed, se, pr, ca, tu)) \rightarrow pr \geq 8) \}$$

SQL equivalente:

```
SELECT c.nombre FROM carrera c WHERE NOT EXISTS (SELECT 1 FROM estudiante e WHERE e.id_carrera=c.id_carrera AND e.promedio < 8) ORDER BY c.nombre;
```

Explicacion: La carrera se conserva si no existe estudiante de esa carrera con promedio menor a 8.

Resultado ejemplo:

nombre
Ingenieria en Inteligencia Artificial

6. Instrucciones de instalacion y entrega

Para ejecutar el proyecto en PostgreSQL, crear una base de datos y ejecutar los scripts en orden: sql/01_schema.sql, sql/02_data.sql, sql/03_consultas_basicas.sql, sql/04_consultas_avanzadas.sql y sql/05_consultas_finales_20.sql. El repositorio incluye README.md con estructura, diagramas, instrucciones y resultados.

```
psql -d practica789 -f sql/01_schema.sql
psql -d practica789 -f sql/02_data.sql
psql -d practica789 -f sql/03_consultas_basicas.sql
psql -d practica789 -f sql/04_consultas_avanzadas.sql
psql -d practica789 -f sql/05_consultas_finales_20.sql
```

7. Conclusiones

La practica permite comprobar que las operaciones fundamentales del modelo relacional pueden representarse en algebra relacional, calculo relacional y SQL. El sistema universitario facilita consultas simples, avanzadas y universales, especialmente con division y cuantificadores. El repositorio queda preparado para entrega, revision y ejecucion de scripts SQL.